Multi-Country Accounts Payable Processing

Arabic Invoice AI Pipeline

Implementation Specification

Ref: `AP-ARABIC-2026-v1.2`

SAP-OSS Accounts Payable Team

April 2026

Abstract

This specification defines the full implementation of Standard Chartered Bank's multi-country Accounts Payable (AP) capability across the Middle East & Pakistan (MEP) footprint, with particular emphasis on the Arabic-language invoice processing pipeline sanctioned by the AI solutioning team under the initiative titled *Arabic Invoice, Peer analysis, & Translation* and announced by Manoj Hemath Kumar on 2025-11-23. Three orthogonal workstreams are specified in full:

- an **Arabic-invoice AI pipeline** that ingests PDF or e-mail attachments, performs bilingual OCR and schema-guided field extraction, applies an Arabic→English translation agent, and hands a validated record to downstream AP;
- a **multi-country VAT compliance engine** that enforces the regulator-specific checklists of ZATCA (Saudi Arabia), ETA (Egypt), NBR (Bahrain), and the Qatari counterparty conventions;
- an **AP workflow and approval engine** that formalises the seven activities of the Bahrain *Accounts Payable Departmental Operating Instruction* (DOI) as explicit state machines.

All three workstreams consume a single canonical data schema (Chapter 2) defined in `structured/schema/`. The specification intentionally *excludes* the downstream SAP/PeopleSoft integration, which is covered by a separate interface document.

Reading order. Chapter 1 situates the work; Chapter 2 defines the normalised record types; Chapter 3 covers the AI pipeline; Chapter 4 covers the VAT engine; Chapter 5 covers the AP workflow engine; Chapter 6 specifies controls, testing, and acceptance; Chapter 7 is a traceability appendix linking every requirement to a source document and a structured record in `structured/`.

Conventions. Arabic fragments are set with the `Geeza Pro` font family, for example هيئة الزكاة والضريبة والجمارك. File paths are typeset in `monospace` and are relative to the repository root. Enum values in running text appear in SMALL CAPS; they are defined authoritatively in `structured/enums.yaml`.

□□□□□□□□ □

□□□□□□□□□ □□□ □□□□□

□.□ □□□ □□□□□□□□

Standard Chartered Bank operates Accounts Payable across a heterogeneous regulatory and linguistic landscape. Depending on the country, an incoming invoice may be:

- issued by a national e-invoicing portal (Egypt's ETA, with a 26-character ELECTRONIC_ID; Saudi Arabia's ZATCA with an analogous assessment reference);
- purely Arabic (as in the two Egyptian invoices `3CPJR5CQ…K10.pdf` and `B1STE8RH…SJ10.pdf` and the Saudi VAT assessment اشعار صدور فاتورة);
- bilingual (Qatar's QCSD / EDAA memo);
- wholly English with pre-assigned GL coding (e.g. the Bahrain DOI-covered PO flow).

Today the Arabic-language material is handled manually: a finance analyst reads the PDF, transcribes key fields into an OTP/VAT checklist (see the Saudi Q3 2025 checklist in `source/Saudi…OTP…`), and routes it through the DOI approval chain. The November 2025 e-mail thread from Manoj Hemath Kumar identifies this manual path as the target for AI replacement.

□.□ □□□□□

The specification targets three outcomes, each with a direct lineage to the sample corpus in `source/`:

1. **Structured ingestion.** Every Arabic or bilingual invoice becomes a record conforming to `structured/schema/invoice.schema.json` (Chapter 2).
2. **Deterministic compliance.** Every invoice is passed through a rule engine that emits a VAT compliance checklist conforming to `structured/schema/vat-checklist.schema.json` (Chapter 4), with the rule set derived from the regulator-specific controls in the corpus.

3. **Auditable approval.** Every payment follows one of the four state machines in `structured/workflows/` and meets the controls enumerated in the Bahrain DOI (Chapters 5 and 6).

[redacted]

[redacted] [redacted] [redacted] [

- Integration contracts with PeopleSoft PSAP/PSGL, eBBS, eOPS, and Tax-Q. These are covered by a separate interface specification.
- Vendor master-data on-boarding and the 48-hour screening service itself; this document assumes a screening result is available as a boolean precondition.
- The encrypted template `Business Instruction Template - V1.0 (002) (1).xlsm` and the duplicate `EDAA…[65].pdf`; neither is referenced by any structured record.
- Peer/competitor analysis across jurisdictions, which the e-mail thread itself identifies as a separate initiative.

1.1 Discovery cadence and scope

Per the e-mail, every country instance of this system follows a five-session agile discovery cadence (Table 1.2); the AI pipeline of Chapter 3 is designed to advance with that cadence.

Table 1.2 Discovery cadence and scope as structured/enums.yaml.

Phase	Sessions	Goal
Discovery	Sessions 1–2	Map business rules, data sources, and exceptions..
Design	Sessions 3–4	Specify prompts, schemas, and validation rules..
Pilot	Sessions 5–6	Run on a representative sample, measure accuracy..
Production	Sessions 7–8	Harden error handling, monitoring, and ops..
Review	Sessions 9–10	Review accuracy against ≥1 representative sample..

[redacted]

[redacted]

[redacted] structured/

The machine-readable corpus is organised as in Table 2.1. All records validate against a JSON Schema Draft 07 file under `structured/schema/`. The `structured/validate.py` script walks the tree, resolves `$ref` entries via a local registry (so offline validation succeeds), and exits non-zero on any failure; it is the authoritative acceptance gate for this chapter.

[redacted] structured/.

[redacted]	[redacted]
`schema/invoice.schema.json`	[redacted]
`schema/vat-checklist.schema.json`	[redacted]
`schema/payment-instruction.schema.json`	[redacted]
`schema/approval-workflow.schema.json`	[redacted]
`schema/doi-policy.schema.json`	[redacted]
`invoices/*.json`	[redacted]
`payment-instructions/*.json`	[redacted]
`vat-checklists/*.json`	[redacted]
`policies/*.json`	[redacted]
`workflows/*.json`	[redacted]
`enums.yaml`	[redacted]
`corpus.json`	[redacted] ↔ [redacted] ↔ [redacted]
`validate.py`	[redacted]

[redacted]

The invoice record is the central object shared by ingestion (Chapter 3), the VAT engine (Chapter 4), and the approval engine (Chapter 5). Table 2.2 enumerates the top-level fields; Listing 2.1 is a worked example taken verbatim from `structured/invoices/eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10.json`.

Invoice ELECTRONIC_ID for ETA must match the regex in `structured/enums.yaml`

□□□□□□□□ □.□□ □□□□ □□□□□□□□ □□□□□□□ □□□ □□□ □□□□□ □□□ □□□□ □□□□□□□□ □eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10□. □□□□□□ □□□□□□□□ □□□ □□□□□□ □□□ □□□□□ □□□□□□ □□□□□□□□ □□□□ □□□.


```
{
  "id": "eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10",
  "schema_ref": "invoice.schema.json",
  "metadata": {
    "country": "EG",
    "tax_authority": "ETA",
    "document_kind": "E_INVOICE",
    "electronic_id": "3CPJR5CQ4PNNN9A4XHBB370K10",
    "issue_date": "2025-07-15",
    "submission_date": "2025-07-15T16:10:00+00:00",
    "language": "ar",
    "status": "VALID"
  },
  "seller": {
    "name_ar": "          ",
    "tax_id": "438212282",
    "tax_id_type": "ETA_TRN",
    "activity_code": "1811"
  },
  "buyer": {
    "name_ar": "           ",
    "tax_id": "670307459",
    "tax_id_type": "ETA_TRN"
  },
  "line_items": [{
    "description_ar": "         42      ",
    "description_en": "Circular Trodat seal 42mm, series",
    "quantity": 1, "unit_of_measure": "EA",
    "unit_price": 580000.00, "line_total": 580000.00
  }],
  "totals": {
    "currency": "EGP",
    "net": 580000.00, "vat_amount": 81200.00,
    "vat_rate_percent": 14, "vat_category": "FI",
    "gross": 661200.00
  },
  "source_document": {
    "path": "docs/arabic/source/3CPJR5CQ4PNNN9A4XHBB370K10.pdf",
    "sha256": "2b9126...4b2c0",
    "ocr_text_path": "docs/arabic/extracted-text/eg-eta-3CPJR...txt"
  }
}
```

[illegible] [illegible] [illegible] [illegible] [illegible] `invoice.schema.json`.

[illegible]	[illegible]	[illegible]	[illegible]
`id`	[illegible]	✓	[illegible] [illegible] [illegible] `invoices/`.
`metadata`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible] [illegible].
`seller`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible] [illegible] `tax_id_type`.
`buyer`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible].
`line_items[]`	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] `description`[illegible] `description_ar`[illegible] `description_en`.
`totals`	[illegible]	✓	`currency`[illegible] `gross` [illegible].
`tax_period`	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible].
`signature`	[illegible]	[illegible]	[illegible] $\in$ [illegible] [illegible] [illegible] [illegible].
`source_document`	[illegible]	✓	`path`[illegible] `sha256`[illegible] `ocr_text_path`.
`notes[]`	[illegible]	[illegible]	[illegible] [illegible] [illegible].

(`^[0-9A-Z]{26}$`); the ZATCA assessment number is a decimal prefix matching the taxpayer's ZATCA_TDN. The `status` enum is {VALID, INVALID, PENDING, CANCELLED}; ZATCA assessments are always VALID on arrival.

[illegible] [illegible] [illegible] [illegible]

The payment instruction record is emitted by the approval engine (Chapter 5) when a validated invoice and approval workflow reach a terminal state. Table 2.3 enumerates the top-level fields of `payment-instruction.schema.json`. Two instance records are provided in the corpus under `structured/payment-instructions/`: the Qatar EDAA annual fee and the QCSD payment instruction referenced in the overview chapter.

[illegible] [illegible] [illegible] [illegible] [illegible] `payment-instruction.schema.json`.

[illegible]	[illegible]	[illegible]	[illegible]
`id`	[illegible]	✓	[illegible] [illegible] [illegible] `payment-instructions/`.
`metadata`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].
`payer`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].
`payee`	[illegible]	✓	[illegible] [illegible] [illegible] [illegible].
`amount`	[illegible]	✓	`value` [illegible] [illegible] `currency` [illegible] [illegible] [illegible].
`payment_method`	[illegible]	✓	[illegible] [illegible] `bank_transfer`[illegible] `check`[illegible] `card`[illegible] `cash`[illegible] `other`.
`reference`	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]`invoice_id`[illegible] `description`[illegible].
`approval_chain`	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].
`due_date`	[illegible]	✓	[illegible] [illegible].
`status`	[illegible]	✓	[illegible] [illegible] `draft`[illegible] `approved`[illegible] `emitted`[illegible] `settled`[illegible] `cancelled`.

[illegible] [illegible] [illegible] [illegible]

The KSA OTP/VAT checklist is encoded as an ordered array of items (Table 2.4). Each item records the regulator's question, the analyst's verdict (Y/N/NA), and the extracted value. The authoritative list is the eleven points in the Saudi Q3 2025 checklist; NBR (Bahrain) maintains an analogous set drawn from Activity 3 of the DOI.

□□□□□ □.□□ □□□□□□ □□ □ □□□ □□□□□□□□□□ □□□□□□.

□□□□□	□□□□	□□□.	□□□□□
period	□□□□□□	✓	start□ end□ label.
checklist[]	□□□□□	✓	□□□□□ □□□□□□□ □…n□ □□□□ □□□ result ∈ □□□□□□□□.
gl_coding	□□□□□□	✓	□□□□□ □□□□□□ □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□.
supplier	□□□□□□	✓	supplier_type ∈ □□□□□□□□□□□□□ □□□□□□□ □□□□□□□□□□ □□□ □□□□□□□□.
tax_type_classification	□□□□□□	✓	□□□ □□□□□□□□□ □□□□.
withholding_tax	□□□□□□	□□	□□□□ □□□ □□□□□□ □□ □□□□□□□□□□□.
psid	□□□□□□	□□	□□□□□□□□ □□□□.

□□□□□□□ □.□□ □□□□□□□ □□□□ structured/vat-checklists/sa-otp-vat-q3-2025.json □□□□□□□ □□□□□ □□□□ □□ □□□ □□□ □□□□□□□□ □□□□□□□□□.

```
"checklist": [
  {"n": 1, "label": "Invoice in Arabic language", "result": "Y"},
  {"n": 2, "label": "Addressed to Standard Chartered Capital", "result": "Y"},
  {"n": 3, "label": "Date of issue", "result": "Y", "value": "2025-10-30"},
  {"n": 4, "label": "Sequential number", "result": "Y", "value": "3003093648253002"
      },
  {"n": 5, "label": "Tax Identification Number (15-digit)", "result": "NA",
     "regulation": "Supplier is ZATCA itself -- government body"}
],
```

□.□ □□□□□□□□ □□□□□□□□□□□□

Derived from the Qatar QCSD instruction. The approval chain is an array of typed signatories; each entry carries PSID, role, and approval timestamp — giving a tamper-evident audit trail once hashed. The `chart_field` is stored as a dash-separated composite string (e.g. `665011-4551-800` = account-product-operating_unit) because this is the convention used by the source memos.

□.□ □□□□□□□□□ □□□□□□□□□

Each workflow record is a deterministic state machine. States are nodes; transitions are edges tagged with a trigger and, where relevant, a `required_role` and a list of `preconditions`. The `delegation_of_authority` array gates roles by LCY threshold and is consulted by the approval engine before any transition out of a "x-approved" state. Four workflow instances are shipped in the corpus:

- `po-eproc` — PO / eProcurement path (DOI Activity 3);
- `manual-one-off` — one-off manual payment (DOI Activity 5);
- `manual-recurring` — recurring manual with dispensation (DOI Activity 5);
- `vat-payment` — authority-counterparty (e.g. ZATCA) path.

[illegible] [illegible]

The Bahrain Accounts Payable DOI is represented as a `doi-policy` record whose `activities[]` array preserves the original numbering (1–7) and whose `controls[]` array gives each control an ID, description, pointer back to the activity, and the risk it mitigates. This is what the approval engine consults to decide *which* workflow instance applies to a given invoice.

[illegible]

The file `structured/enums.yaml` is the single source of truth for controlled vocabulary used across the corpus: country codes, tax-authority codes and portal URLs, VAT category codes, `tax_id_type` regexes, business-unit codes, approval-role codes, payment methods, and the five AI-session phase codes. Any new enum value must be added to `enums.yaml` *before* being used in a schema or record; `validate.py` enforces this.

[illegible]

Each record stores the SHA-256 of its source document in `source_document.sha256`. Reproducing the corpus from the originals is therefore a single command:

```
find docs/arabic/source -type f -print0 \
  | xargs -0 shasum -a 256 \
  | diff - <(jq -r '.items[] | select(.sha256) | "\(.sha256)  \(.source_path)"' \
              docs/arabic/structured/corpus.json | sed 's|docs/arabic/source/||')
```

A clean diff proves the corpus is unaltered since the last specification snapshot.

[illegible] [illegible] [illegible]

As of 18 April 2026, all schemas in this specification are published in the unified schema registry at `docs/schema/`. Migration status is *complete* (see `docs/schema/registry.json` field `migration.status`). This consolidation addresses cross-document schema divergence identified in the specification review.

[illegible]. `docs/schema/registry.json` — Master index of all schemas across domains.

[illegible]. `docs/schema/common/base-types.schema.json` — Shared definitions (`Metadata`, `Currency`, `TaxIdentifier`, `EntityParty`) referenced via JSON Schema `$ref`.

[illegible]. `docs/schema/common/enums.yaml` — Single source of truth for enumeration values including `countries`, `vat_category`, `supplier_type`, and `vat_thresholds` used by this specification.

□□□□□□ □□□□□□□. `docs/schema/arabic/` — Domain-specific schemas: `invoice.schema.json`, `vat-checklist.schema.json`, `vendor.schema.json`, and `ocr-result.schema.json`. All marked `status: stable`.

The per-country VAT thresholds (e.g. BHD 500 for Bahrain SI/FI boundary) are defined in `docs/schema/common/enums.yaml\#/vat\_thresholds` rather than in domain-specific `enums.yaml` files. Validation:

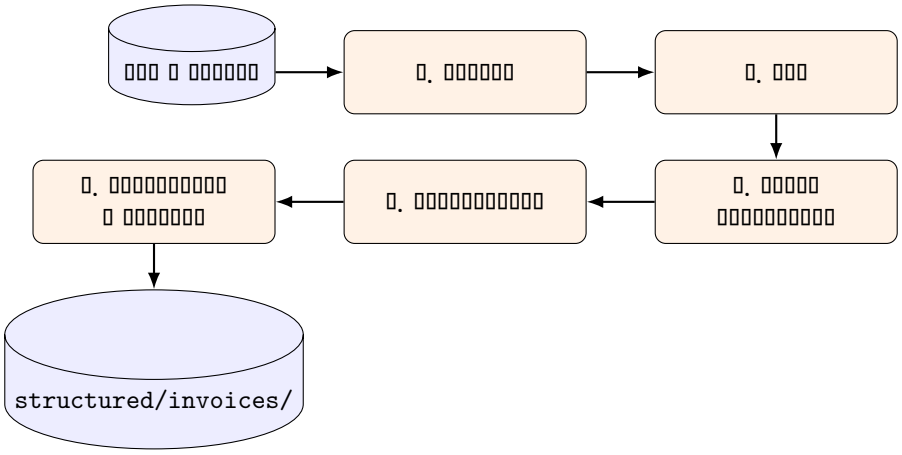
```
python3 docs/schema/validate.py --registry-check
python3 docs/schema/validate.py --domain arabic \
    --schema invoice.schema.json --file invoice.json
```

CI runs `--registry-check` on every pull request touching `docs/schema/` or any dependent pipeline module.

□□□□□□□□ □

□□□□□□ □□□□□□□□ □□ □□□□□□□□□

□.□ □□□□□□□□□ □□ □ □□□□□□



□□□□□□ □.□□ □□□ □□□□ □□□□□□□□□ □□□□□□□□ □□□□ □□□□□ □ □□□□□□□□□□□ □□□□□□□□ □□ □□□ □□□□. □□□□ □□□□□ □ □□□□□□ □□ □□□ □□□□□□□□□□ □□□□□□.

□.□ □□□□□ □ □□ □□□□□□

□□□□□□. A source file, either:

- □ a PDF dropped into a watched folder, or
- □ an e-mail attachment (e.g. the thread captured in `source/FW- AI _ Arabic Invoice….eml`).

□□□□□□□. A new entry staged for Stage 2 and a provisional `source_document` object:

```
{ "path": "docs/arabic/source/<file>", "sha256": "<sha>" }
```

The SHA-256 is computed first and compared against `structured/corpus.json`; a match short-circuits re-processing (idempotence) and is the duplicate-payment control (§6).

□□□□□□□□□□□□. The MIME detector must classify the file as one of `application/pdf`, the OOXML Word type (`application/vnd.openxmlformats-officedocument.wordprocessingml.document`), `text/plain`, or `message/rfc822`. All other types are rejected with reason UNSUPPORTED_MEDIA.

□.□ □□□□□ □ □□ □□□

□□□□□□□. The corpus has been produced with:

```
pdftoppm -r 300 <input.pdf> work/page -png
for img in work/page-*.png; do
  tesseract "$img" stdout -l ara+eng >> <output>.txt
done
```

The combined `ara+eng` language model is mandatory: invoices routinely contain English brand names, Latin-script SKU codes, and numerals alongside Arabic prose.

□□□□□□□ □□□□. Stage 2 emits a per-page confidence metric (mean of tesseract's `conf` values over non-empty tokens). Pages below **55%** confidence are flagged for human review; below **30%** the file is rejected with reason OCR_LOW_CONFIDENCE and routed to manual processing.

□□□□□□□□□□□. The 30 %/55 % thresholds are preliminary estimates pending production pilot validation.

v1 status: These values were derived from three golden-set documents, which is insufficient for statistical confidence. The 30 % threshold represents the lowest observed confidence on pages that still yielded correctable records; 55 % is the lowest value achieving error-free extraction. **Both thresholds are TBD pending pilot:** production calibration requires ≥50 documents per country before these values can be considered validated. The pilot will also establish per-jurisdiction calibration as OCR quality varies with document source (e.g., ZATCA assessments vs. commercial invoices).

□□□□□□□□. The raw OCR text is written to `extracted-text/<slug>.txt` and its path is recorded in `source_document.ocr_text_path`. This preserves the training signal for future model refinement.

□.□ □□□□□ □ □□ □□□□□□□□□□□□□□ □□□□□ □□□□□□□□□□□

□□□□□□□□□. Produce a candidate record conforming to `invoice.schema.json` from the OCR text. The extraction is *schema-guided*: the LLM prompt is constructed at run time by serialising the schema (with `description` fields) so the model is told exactly which JSON keys it may emit. This is the single largest lever against hallucinated fields.

□□□□□□ □□□□□□□□□. Pseudocode; the concrete implementation uses the Anthropic Messages API with structured-output tooling.

□□□□□□□ □.□□ □□□□□□□□□□□□□□ □□□□□□□□□□□ □□□□□□ □□□□□□□□□.

```
SYSTEM: You extract tax-invoice fields into a JSON object that MUST
        validate against the provided JSON Schema. If a field is not
        present in the source, omit it -- NEVER invent a value.
USER:   <schema>
        ---
        OCR TEXT (Arabic may appear in RTL order; preserve spelling):
        <ocr_text>
        ---
        Emit JSON only. Set "metadata.language" to "ar", "en", or "ar+en"
        based on the dominant script.
```

□□□□□□□□□□□ □□□□□□. The following fields are the most error-prone in the shipped corpus and must be handled with care:

□□□□□ □.□□ □□□□□□□□□□□ □□□□□□□□□ □□□□□□□□□ □□ □□□ □□□□□□ □□□ □□□□□□□□□□□□.

□□□□□	□□□□□□□	□□□□□□□□□□
electronic_id	□□□□□□□□□□ □□□□ □□□ □□□□□ □□□□□ □□ □□□□□ □□ □□□□□ □□□□□ □□□□□□□ □□.□. B1STE8→BISTE8□.	□□□□□□□□□ □□□□□□□ □□□ □□□□□ ^[0-9A-Z]{26}$□ □□ □□□□□□□□□□ □□□□□□ □□ □□□ □□□ □□□□ --psm 6.
totals.gross	□□□□□□ □□□□□□ □.١٢٣٤٥٦٧٨٩□ □□□ □□□□ □□□□□□□ □□□□□□□□.	□□□□□□□□□□□□□□ □□□□□□□□□□ □□□□□□□□□□□□□ □□□□□□ □□□□ □□□□□ □□□□□□ □□□□□□□.
vat_rate_percent	□□□□□□□□□□□ □□ □□□□□□ □□□□ □□□□ 0.	□□□□□□□□□□□□□ □□□□ vat_category□ □□ □□ □□ □□□□□ □□□□ □□□□□□□□□□ □□□□□□□□ □□□□ □□□□□□.
signature.kind	□□□□□ □□□□□□□□□□□□ □□□□ □□ □□□□□□□□□□ □□□□□.	□□□□□□□ □□ □□□□□□□□□□□□□□□□□□ □□□ document_kind □ □□□□□□□□□□□□□□□.
seller.name_en	□□□□□□□□□□□□ □□□□□□□□□ □□□□ □□ □□□□□□□ □□□□.	□□□□□ □□□□□□ □□□□□ □ □□□□ □□□□□□□□□ □□□ □□□□□□□□□□□□.

□□□□□□. A candidate record is written in memory only (never persisted yet) and passed to Stage 4.

□.□ □□□□□ □ □□ □□□□□□→□□□□□□□ □□□□□□□□□□□□

□□□□□. This stage does *not* translate the whole document. It translates only the free-text fields that downstream AP actually uses in English:

- □ `seller.name_en` and `buyer.name_en` if absent;
- □ every `line_items[].description_en` if absent;
- □ `seller.address_en` and `buyer.address_en`.

Tax IDs, amounts, dates, and enum values are *never* passed through translation; they are copied verbatim.

□□□□□□□□. A fixed glossary enforces terminology consistency. Worked examples:

□□□□□ □.□□ □□□□□□ □□□□□□□□. □□□ □□□□□□□□□□□ □□□□□□ □□ □□□□ □□□ □□□□□□□□□□□□ □□□□□ □□□□ □□□□□□.

□□□□□□	□□□□□□□ □□□□□□□□
هيئة الزكاة والضريبة والجمارك	□□□□□□ □□□ □□□ □□□□□□□ □□□□□□□□□ □□□□□□□
بنك ستاندرد تشارترد	□□□□□□□□ □□□□□□□□□ □□□□
ضريبة القيمة المضافة	□□□□□□□□□□□ □□□ □□□□□
اشعار صدور فاتورة	□□□□□□□ □□□□□□□□ □□□□□□□□□□□□
فاتورة ضريبية	□□□ □□□□□□□

□□□□□□□ □□□□. Back-translation is run as a sanity check: the English output is translated back to Arabic and compared to the source Arabic string. The metric is the *normalised Levenshtein distance* $d = \mathrm{lev}(a,b)/\max(|a|,|b|)$ computed at the Unicode-codepoint level after Arabic NFKC normalisation (stripping tatweel and harakat). The 0.30 ceiling was chosen because the three golden-set Arabic strings have pairwise $d \leq 0.18$ between their recorded Arabic and a human-verified back-translation, leaving a $\geq 1.5\times$ safety margin. Distances above **0.30** are flagged (TRANSLATION_DRIFT) and the original Arabic is retained as authoritative.

□.□ □□□□□ □ □□ □□□□□□□□□□ □□□ □□□□□□□

□□□□□□□□□□. The candidate record is validated with `validate.py` (Chapter 2). Any schema failure halts the pipeline and the record is written to a side-queue (`structured/invoices/rejected/`) with a `notes[]` entry summarising the failure.

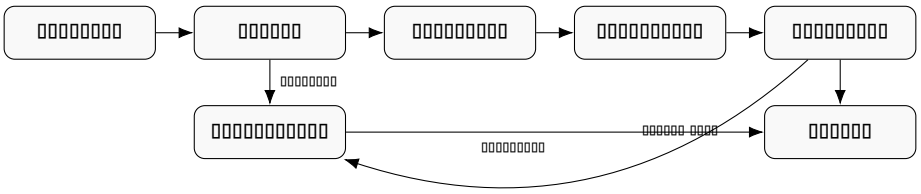
□□□□□□□. On success, the pipeline consults the structured `bh-accounts-payable-doi.json` and the `workflows/` set to decide which state machine to instantiate:

1. If the counterparty is a tax authority (`supplier_type` = GOVERNMENT or `tax_authority` = ZATCA) ⇒ `vat-payment` workflow.
2. Else if there is a PO reference in `notes[]` ⇒ `po-eproc` workflow.
3. Else if a prior `manual-recurring` record exists for the same `vendor_id` ⇒ `manual-recurring` workflow.
4. Otherwise ⇒ `manual-one-off` workflow.

□□□□□□. The record is written to `structured/invoices/<slug>.json` and a new item is appended to `corpus.json`. This is the only stage in the pipeline that mutates the corpus.

□.□ □□□□□□□□□ □□□□□□□ □□□□□

Figure 3.2 models the pipeline itself as a small state machine; each invoice occupies exactly one of these states at any moment. The control plane exposes three operations: `ingest(file)`, `retry(id)`, `release(id)`.



□□□□□□ □.□□ □□□□□□□□ □□□□□□□□□□□□□□ □□□□□□ □□□ □ □□□□□□ □□□□□□□□.

□.□ □□□□□□□□□□□ □□□□□□□□

□□□□□□ □□□. The three Arabic records in `structured/invoices/` form the regression golden set. Every code change must re-produce byte-identical records from the source PDFs; `validate.py` plus a `jq`-driven equality check is the verifier.

□□□□□□□□□□ □□□□□□□□. For each round of Session-5 testing the harness reports:

- **Per-field exact-match accuracy** (over the 27 scalar fields in `invoice.schema.json`).
- **Per-field F1** on `line_items[]` treated as sets (descriptions matched after glossary-aware fuzzing).
- **Human-approval rate.** Fraction of records released by HITL with zero edits. Target ≥ 0.75 after Session 4, ≥ 0.90 after Session 5.

□□□□□□□□ □□□□□□□□□□□. The five-session model from Table 1.2 is scheduled per country; the `ai_session_phases` enum in `enums.yaml` is the system of record. An AI rollout is complete for a country once the Session-5 pass meets the human-approval-rate target against a cohort of ≥ 5 stakeholders.

[illegible] 6

[illegible] [illegible] [illegible]

6.1 [illegible]

Given a validated invoice record, the VAT engine must:

1. select the correct regulator and ruleset from the record's `metadata.country` and `metadata.tax_authority`;
2. run every applicable rule, producing a `vat-checklist.schema.json` record with one item per rule;
3. decide a final `tax_type_classification` (VAT category code from `enums.yaml`);
4. compute any adjustments (withholding, reverse-charge accrual);
5. refuse to release the record for payment unless every mandatory rule emits Y or NA.

Rules are declarative data, not code: the engine is a rule interpreter. This isolates regulation change from engineering change.

6.2 [illegible]

A rule is a tuple (c, a, p, q, r) where:

- c — country (SA, EG, BH, QA);
- a — tax authority (ZATCA, ETA, NBR, QTA);
- p — a predicate over the invoice record (expressed as a JSON-Logic expression or a named helper);
- q — the checklist *question* emitted when the rule fires (human-readable string);
- r — a `result` $\in$ {Y, N, NA} computed by evaluating p and optionally a secondary predicate.

The engine iterates over the ruleset for (c, a) in a deterministic order and appends each evaluated rule as an item in the output `checklist[]`. The final `tax_type_classification` is chosen by a *decision function* that maps the full result vector to a single VAT category.

4.4 [illegible]

4.4.1 [illegible]

The eleven-point ruleset is lifted verbatim from the Q3 2025 checklist and is shown in Table 4.1. Each row is one rule; the mapping from rule to `result` is given in the predicate column.

[illegible] 4.1 [illegible] [illegible] [illegible]

```
    classification = FI
elif totals.vat_rate_percent in {5, 10, 7, 14, 20}:
    # A rate that is legal in another jurisdiction but not in KSA.
    # Never silently classify: emit HOLD + human review.
    classification = OTHER
    verdict        = HOLD        # consumed by the AP engine
    reason         = "unexpected_vat_rate_for_ksa"
else:
    # Unknown rate (e.g. 3%, negative, null): always HOLD.
    classification = OTHER
    verdict        = HOLD
    reason         = "unknown_vat_rate"
```

[illegible]

The Saudi Q3 2025 assessment (اشعار صدور فاتورة) has `supplier.supplier_type` = GOVERNMENT and `tax_authority` = ZATCA; the decision function correctly yields `tax_type_classification` = RC. All eleven rules fire; rules 5 and 11 are NA because the counterparty is ZATCA itself and the buyer is KSA-resident. The total `gross` = SAR 1,558,629.96 (due 2025-10-31) is posted as a reverse-charge VAT payment through the `vat-payment` workflow.

[illegible]

[illegible]

[illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible] [illegible]
[illegible]	electronic_id [illegible]	[illegible] ^[0-9A-Z]{26}$.
[illegible]	[illegible] [illegible] [illegible]	seller.tax_id [illegible] ^[0-9]{9}$.
[illegible]	[illegible] [illegible] [illegible]	buyer.tax_id [illegible] ^[0-9]{9}$.
[illegible]	[illegible] [illegible] [illegible]	seller.activity_code [illegible].
[illegible]	[illegible] [illegible] [illegible] [illegible]	submission_date − issue_date ≤ [illegible] [illegible].
[illegible]	[illegible] [illegible] [illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].
[illegible]	[illegible] [illegible]	[illegible] [illegible] line_total [illegible] totals.net.

[illegible]

```
if seller.activity_code in EXEMPT_ACTIVITIES:
    classification = EX
elif totals.vat_rate_percent == 14:
    classification = FI
elif totals.vat_rate_percent == 0:
    classification = ZR
else:
    classification = OTHER
```

The set `EXEMPT_ACTIVITIES` is a configured lookup. Activity `8890` (“Professional services”, as used by the Chapter Zero corporate-membership invoice) is included.

[redacted]

[redacted] `3CPJR5CQ…K10`. All seven rules pass; activity code `1811` is not exempt; `totals.vat_rate_percent = 14`; the decision function yields FI (Full Tax Invoice). Gross `EGP 661,200` posts to the PO e-Proc or manual path depending on the presence of a PO reference.

[redacted] `B1STE8RH…SJ10`. Activity code `8890` is in the exempt set; rule 6 still passes because 0% with an exempt code is consistent; decision yields EX. No VAT accrues and the routing is `manual-one-off`.

[redacted]

[redacted]

Derived from Activity 3 of the Bahrain DOI (`structured/policies/bh-accounts-payable-doi.json`). The NBR ruleset is wider than KSA’s eleven points because the DOI distinguishes ten VAT invoice types.

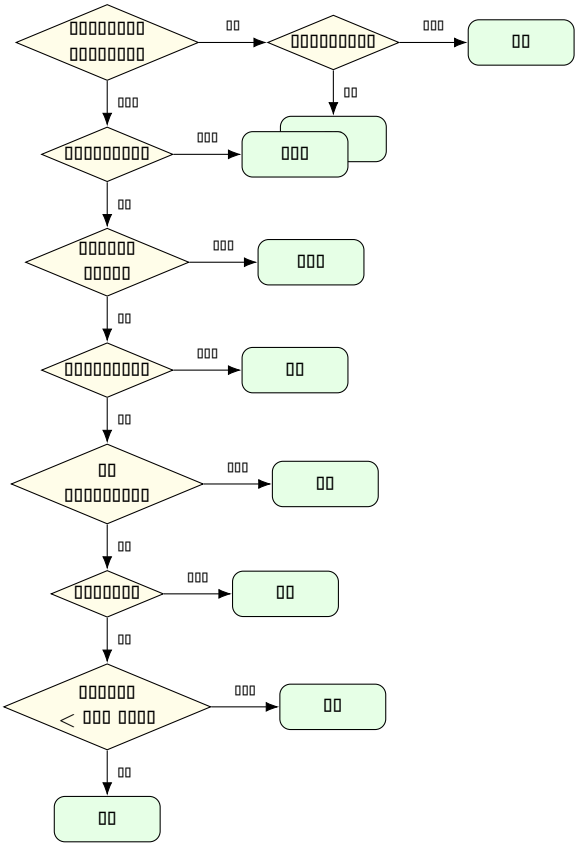
[redacted] `vat_category` [redacted].

[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted] $\geq$ [redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].
[redacted]	[redacted]	[redacted].

[redacted]

The DOI fixes BHD 500 as the SI/FI threshold for Bahrain. This value is stored in the per-country engine configuration file `vat-thresholds.yaml` (not in `enums.yaml`, which holds only enum *types*). The threshold structure is:

```
thresholds:
  BH:
    si_fi_boundary: 500
    currency: BHD
```



예외처리 0.00 예외처리를 위한 비즈니스 로직입니다. 이미 등록된 사업자가 아니라면 국세청 조회를 거쳐 vat_category 값을 결정하는 흐름입니다.

Regulator updates are applied by editing this file; a CI check validates that all threshold values match the authoritative DOI documents.

6.4 Qatar VAT

Qatar does not currently impose VAT. The engine therefore runs a single rule — *payment is to a valid counterparty with a QCSD-registered account* — and classifies the record as OTHER unless a future VAT regime is detected (`metadata.tax_authority` ≠ QTA). The `structured/payment-instructions/qa-qcsd-payment-instruction.json` record is the reference.

6.5 Withholding tax

Withholding rules are modelled as a secondary ruleset keyed on *supplier residence*. The engine emits a `withholding_tax` object with `applicable`, `rate_percent`, and `amount`. KSA-resident suppliers yield `applicable = false` (NA), mirroring the Q3 2025 checklist. Non-resident, non-GCC suppliers trigger a country-specific rate (e.g. 5% KSA royalty, 15% services) that must be explicitly configured in `enums.yaml` before the engine can emit Y/N.

6.6 Outputs

For every invoice the engine produces:

- a `vat-checklist` record stored at `structured/vat-checklists/<slug>.json`;
- an updated `invoice` record with a populated `totals.vat_category`;
- a verdict of RELEASE or HOLD consumed by the approval engine (Chapter 5).

A hold is produced whenever any rule resolves N. The engine must never up-grade a HOLD to RELEASE without a human recorded in the corresponding workflow transition.

□□□□□□□□ □

□□ □□□□□□□□□ □□□ □□□□□□□□□ □□□□□□

□.□ □□□□□□ □□□□□□□□□□□□□□□□□□

Given a validated invoice record and the VAT engine's verdict (Chapter 4), the approval engine must:

1. select the correct workflow from `structured/workflows/`;
2. instantiate the state machine with the record;
3. drive transitions as approvals arrive (human or system);
4. at every transition, verify the `delegation_of_authority` is respected for the current LCY amount;
5. on reaching a terminal state, emit the payment instruction conforming to `payment-instruction.schema.json`.

The engine is a *pure* workflow executor; it never invents fields.

□.□ □□□□□□□□□ □□□□ □□ □□□ □□□□□□□□ □□□

Table 5.1 is the unabridged activity map from `structured/policies/bh-accounts-payable-doi.json`; the engine maps each activity onto one or more workflow paths.

□.□ □□□□□□□□□ □□□□□□□□□□

□.□ `po-eproc` □□□□□ □□□□□□□□

□□□□□□□□□□□□□□.

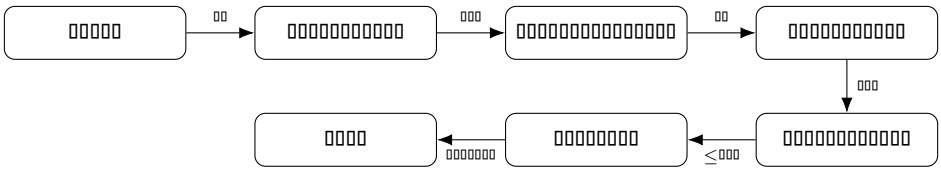
- `tax_q_review` → `posted`: VAT category assigned and FCY revalued at central-bank rate on `issue_date`.
- `posted` → `paid`: LCY amount within DOA per GDAM.

□□□□□ □.□□ □□□ □□□□□ □□□ □□□□□□□□□□□ □□□ □□□ □□□□□□□□□ □□□□□ □□□□ □□□□□.

□	□□□□□□□□□	□□□□□□□□□□□□ □□□□□□□□□□□□	□□□□□□□□□
□	□□□□□□□□□□□□□□	□□□ □□□□□□□□ □ □□□□□□ □□□□□□□ □□□□□.	□□□
□	□□□□□□ □□□□□□□□	□□□□□□□□ □□□□□□□□□□□□□□

□.□ manual-one-off □□□□□ □□□□□□□□

Figure 5.3 captures Activity 5's one-off branch: evidence of one-off nature, vendor screening refreshed within 48 hours, MD and CFO approvals, and account coding captured in the approval document.



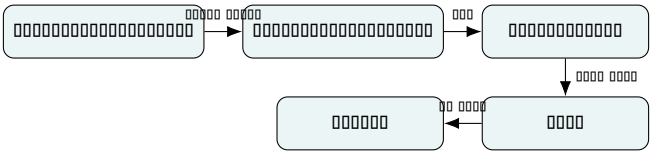
□□□□□□ □.□□ `manual-one-off` □`structured/workflows/manual-one-off.json`□.

□.□ manual-recurring □□□□□ □□□□□□□□

□□□□□□□□□□□□□. A written dispensation from the policy owner is *mandatory* before entry. This is enforced by the engine: the record cannot leave DRAFT until a `dispensation_document_path` is recorded in `notes[]`. Post-payment, the record must reach QUARTERLY_REVIEWED within the following quarter, triggered by the CFO's quarterly log retrieval.

□.□ vat-payment □□□□□ □□□□□□□□

Used when the counterparty is a tax authority (`supplier.supplier_type` = GOVERNMENT). The KSA Q3 2025 assessment walks this workflow from ASSESSMENT_RECEIVED → CHECKLIST_VALIDATED (upon 11/11 rules resolved) → CFO_APPROVED → PAID → POSTED.



□□□□□□ □.□□ `vat-payment`□ □□□□□□ □□□□ □□□□□□ □□ □□□ □□□□□ □□ □□□□ □□□□□□□□□□□. □□□□□ □□□□□□□□ □□□□□ □□□ □□□□□□□□□□ □□□□ □□ □□□□□□□□ □□ □ □□ □□.

□.□ □□□□□□□□□□□□□□□□□□□□□□□□□□

The engine enforces DOA as a linear scan of the workflow's `delegation_of_authority` array, sorted ascending by `max_amount`. The role required is the first entry whose `max_amount` is ≥ the invoice's LCY value. An FX conversion (at the central-bank rate on `issue_date`) is applied before comparison.

Table 5.2 shows the shipped thresholds.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible] [illegible]	[illegible] [illegible]
po-eproc	[illegible]	[illegible]	[illegible]
po-eproc	[illegible]	[illegible]	[illegible]
po-eproc	[illegible]	[illegible]	[illegible]
manual-one-off	[illegible]	[illegible]	[illegible]
manual-one-off	[illegible]	[illegible]	[illegible]
manual-one-off	[illegible]	[illegible]	[illegible]
manual-recurring	[illegible]	[illegible]	[illegible]
vat-payment	[illegible]	[illegible]	[illegible]

[illegible] [illegible]

The shipped workflows carry an `exceptions[]` array. The most important entry is the GSAM legal/professional fees exception (`manual-one-off`): these are *not* covered by the default GDAM approval limits and must be routed to a GSAM-specific approver. The engine consults `exceptions[]` before applying DOA.

[illegible] [illegible] [illegible]

Activity 7 is operated as a *batch* procedure; the approval engine does not execute it per invoice. Reconciliation compares the PSGL extract against the eBBS extract on the accounts listed in the DOI:

- DBU BHD suspense: `09-0906743-50`
- DBU USD suspense: `09-0906743-50`

The aging escalation grid (30/60/90 day buckets) drives automated e-mails to the DBU finance manager; items aged >90 days escalate to the CFO.

[illegible] [illegible]

Every transition records a structured event: `(workflow_id, from, to, actor_psid, timestamp, evidence_refs)`. The concatenation of events is a tamper-evident log because each event hashes its predecessor. This is what downstream auditors consume; it is *not* part of the record schema but sits alongside the `structured/` corpus in an append-only `events/` stream. The event contract specified here is in scope (Chapter 6, “Audit event log”); the storage substrate, retention policy, and restoration runbook are owned by GBS APU Audit and documented in *SCB-APU-AUDIT-001*.

□□□□□□□□ □

□□□□□□□□□□ □□□□□□□□□ □□□ □□□□□□□□□□□

□.□ □□□□□□□□ □□□□□□□□□□

The Bahrain DOI's seven controls (`structured/policies/bh-accounts-payable-doi.json`, `controls[]`) are reproduced in Table 6.1 with the engine mechanism that enforces each.

□□□□□ □.□□ □□□□□□□□ □□□□□□□□□□ □□□ □□□□□□□□□□□□ □□□□□□□□□□□.

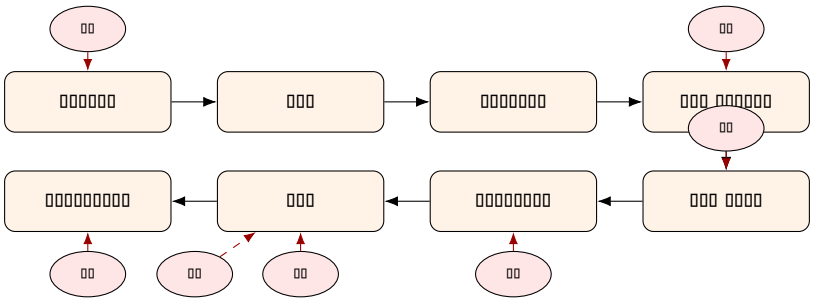
□□	□□□□□□□□□	□□□□□□□□□□□□	□□□□□□□□□ □□
□□	□	□□□ □□□□□□□□□ □□□□□□□□□□ □□□□□□□□ □□□□□□□ □□□□ □□□□ □□□□□□□□.	□□□□□□□□□ □□□□□□□ □□□□□□□ □□□□□□□□□□□□ □□ □□□□ □□□□□ □□□□□ □□□□□□□□□□□□□□□ □□□□□□□□□ □□□ □□□□□.
□□	□	□□□□□□□ □□□□□□□□□□ □□□□□□□□□□□□ □□□□□□□ □□ □□□□□ □□ □□□□□□□□.	□□□□□□□□□□□□□ □□ `cfo_approved`→□□□□□□□□□ □□□□□□□□□□□ □`manual-one-off`□.
□□	□	□□□□□□□ □□□□□□□□ □□□□□□□ □□□ □□□ □□□□.	□□□□□□□ □□□ □□□□ □□□□□□□ □□□ □□□□□□□□□ □□□□□ □□□□□□□.
□□	□	□□□□□□□□□□□□□ □□ □□□□□ □□□ □□□□□□□□□□ □□□□□□□ □□□□□□□□□.	□□□□□□□□□□□□□ □□ □□□□□→□□□□□□□□□□□□□□□□□ □□ `manual-recurring`.
□□	□	□□□□□□□□ □□□□↔□□□□ □□□□□□□□□□□□□□□□ □□□□□□□□ □□□□□□□ □□□□□ □□□□□□□□□□□.	□□□□□ □□□□ □□□□□□□□□□□ □□□□ □□□□□□□□ □□□ □□ □ □□□.
□□	□	□□□ □□□□□□□□□ □□□□□□□□□ □□ □□□□□□□□□□□□□□ □□□□ □□ □□□□□□□□ □□□□.	□□□ □□□□□□□ □□□□ □□□→□□□ □□□□□□□ □□□ □□□□□.
□□	□	□□□□□□□□□□□□□□□□□□□ □□□□□.	□□□□□□□ □□□□□ □ □□□□□□□□□□□□□□□□ □□ □□□□□□□□□ `source_document.sha256`.

□.□ □□□□□□□□ □□□□□□□□□

Testing follows the five-session engagement model (Table 1.2). For each country rollout:

□□□□□□□□ □ □□□ □□□□□□□□□□□. Collect ≥5 sample invoices (regulator-issued if available), normalise into the corpus, confirm per-country ruleset is complete.

□□□□□□□□ □ □□□ □□□□□□□. Walk the end-to-end pipeline on paper using one sample invoice. Identify OCR pitfalls (Table 3.1) specific to the country.



□□□□□□ □.□□ □□□□□□□ □□□□□□□□□□□ □□□□□□ □□□□□ □□□ □□□□□□□□□□□ □□□□□□□ □□□□□□□□□□. □□□□□□ □□□□□□ □□□□ □□□□□ □□□□ □□□ □□□□□□□□ □□□□□□ □.□□ □□ □□□□□□□□□□.

□□□□□□□ □ □□□ □□□□□□. Freeze the rule precedence and the decision function for the VAT engine (§4).

□□□□□□□ □ □□□ □□□□□□□□□□. Run the full pipeline on the cohort; review each extracted record with the local finance team. Human-approval rate target ≥ 0.75.

□□□□□□□ □ □□□ □□□□□□□. Independent run with ≥ 5 stakeholders; target ≥ 0.90 human-approval with zero-edit rate measured.

□.□ □□□□□□□□□□□ □□□□□□□ □□□

The three Arabic records already in `structured/invoices/` (Egypt ETA $\times$ 2, Saudi ZATCA $\times$ 1) together with the KSA checklist and the two Qatar payment instructions constitute the regression golden set.

□□□□□□□□□□□. Every change to the pipeline must preserve:

1. byte-identical `source_document.sha256` for every corpus record (source files are immutable);
2. 100 % schema validity under `validate.py`;
3. identical `tax_type_classification` for every invoice record;
4. identical selected workflow for every record.

□□□□□□□. `validate.py` plus the following `jq` equality check is the regression harness:

```
# Per-field equality of every golden-set record between a reference
# snapshot and the pipeline output.
diff \
  <(jq -S . docs/arabic/structured/invoices/*.json) \
  <(for p in docs/arabic/source/*.pdf; do
      ./pipeline extract "$p"
    done | jq -S .)
```

A non-empty diff is a regression.

[illegible] [illegible] [illegible] [illegible]

[illegible] [illegible] [illegible].

- Corpus integrity check passes (every `corpus.json` path exists; every SHA-256 matches).
- For every source PDF: OCR confidence $\geq$ 30% on every page, schema-valid record emitted.
- Translation back-translation distance $\leq$ 0.30 on every non-empty `description_en`.

[illegible] [illegible] [illegible].

- KSA: all 11 checklist rules evaluated; decision function emits expected VAT category for every golden invoice.
- Egypt: `EXEMPT_ACTIVITIES` lookup table complete for activities present in the corpus.
- Bahrain: each of the 11 NBR invoice types has a corresponding enum code.

[illegible] [illegible] [illegible].

- Every workflow record loadable and schema-valid.
- DOA scan deterministic under round-trip FX conversion.
- Workflow-selection decision tree (Figure 5.1) yields the documented workflow for every corpus record.

[illegible] [illegible]

Table 6.2 links each specification section to the source document that justifies it and to the structured record that realises it. Every row must be present for a full audit pass.

[illegible] [illegible]

A country is declared *production-ready* once:

1. the regression golden set runs clean for 30 consecutive days in the UAT environment;
2. Session 5 acceptance meets the human-approval-rate target (Chapter 3);
3. every VAT rule for the country is present in the ruleset and has at least one passing test case;
4. the country CFO signs off on the workflow configuration, recorded as a `CFO` role entry on the `vat-payment` or `manual-one-off` workflow golden record.

[illegible] [illegible] [illegible]

[illegible]. Three are in scope:

□□□□□ □.□□ □□□□□□□□□□□□□ □□ □□□□□□□□□□□□□□ □□ □□□□□□ □□□□□□□□□ □□□ □□□□□□□□□□□ □□□□□□.

□□□□□□□	□□□□□□□□□□□	□□□□□□	□□□□□□□□□□□ □□□□□□
□□□□□□□ □	□□□□□□□□□ □□ □□□□□	□□□□□ □□□□□□□ □□□□□□□□□□	enums.yaml □□□□□□□□□□□
□□□□□□□ □	□□□□□□□□□ □□□□□□□ □□□□□□	□□ □□□ □□□□□ □□□□□ □□□	schema/invoice.schema.json
□□□□□□□ □	□□□□□□□□ □□□□□□□□□ □□□□□	□□□□□ □□□□□□□ □□ □□□□	schema/vat-checklist.schema.json
□□□□□□□ □	□□□ □□□□□□□ □□□□	□□□ □□□□□	extracted-text/*.txt
□□□□□□□ □	□□□□□□□□□□□ □□□□□□□□	اشعار صدور فاتورة □□□□□.□□□	invoices/sa-zatca-ishaar-005.json
□□□□□□□ □	□□□ □□□□□ □□□□□□□	□□□□□ □□□□□□□ □□ □□□□	vat-checklists/sa-otp-vat-q3-2025.json
□□□□□□□ □	□□□ □□□□□□□ □□ □□□□□	□□ □□□ □□□□	enums.yaml □□□□□□□□□□□□□□□□□.□□□□
□□□□□□□ □	□□□ □□ □□□□□□□ □□□□□	□□ □□□ □□□□□□□□ □	policies/bh-accounts-payable-doi.json
□□□□□□□ □	□□□□□□□□□□ □□□□	□□ □□□	policies/bh-accounts-payable-doi.json
□□□□□□□ □	po-eproc □□□□□ □□□□□□□	□□ □□□ □□□□□□□□ □	workflows/po-eproc.json
□□□□□□□ □	manual-one-off □□□□□ □□□□□□□	□□ □□□ □□□□□□□□ □ □□□□□□□□□	workflows/manual-one-off.json
□□□□□□□ □	manual-recurring □□□□□ □□□ □□□□□	□□ □□□ □□□□□□□□ □ □□□□□□□□□□□	workflows/manual-recurring.json
□□□□□□□ □	vat-payment □□□□□ □□□□□□□	□□□ □□ □□□□ □□□□ □ □□□□□ □□□□□□□ □□□□	workflows/vat-payment.json
□□□□□□□ □	□□□□□□□□ □□□□□□	□□ □□□ □ □□□ □□□□□□□□□ □ □□□□□□□□ □□□□□	policies/bh-accounts-payable-doi.json

□□□. Developer workstations and CI; reads the corpus from the Git working tree. No live invoices.

□□□. Shared test environment reading a frozen copy of the corpus; live DOI activity stops at CFO_APPROVED (no SAMA/SWIFT rail). Country finance and GBS APU run Session 4/5 here.

□□□□. Live environment; the only one authorised to transition to PAID.

□□□□□□□□ □□□□□□. Each promotion requires a two-party approval recorded as an event in the `events/` stream (Chapter 5):

Transition	Approvers	Evidence
DEV→UAT	Tech lead + QA lead	Green CI build, clean regression.
UAT→PROD	Country CFO + GBS APU head	Session 5 acceptance report; 30-day UAT stability; CFO workflow golden record.

□□□□□□□□. Deployments are blue/green. The engine, rules, and schemas are versioned with a content-addressed tag (SHA-256 of `structured/corpus.json` at release time). Rollback is a one-command revert to the previous tag; no database migration is required because the canonical corpus is append-only. Any in-flight invoice whose workflow record references a schema version that was rolled back is automatically held in AWAITING_REVIEW.

[illegible]. Ruleset edits (Chapter 4) and workflow edits (Chapter 5) are routed through the same two-party sign-off; schema edits require an additional sign-off from the AI pipeline owner because they invalidate the regression golden set (§3).

[illegible] [illegible] [illegible]

The approval engine emits an append-only event per transition (`workflow_id`, `from`, `to`, `actor_psid`, `timestamp`, `evidence_refs`), hashing its predecessor so the stream is tamper-evident. The stream is *in scope* for compliance purposes but the storage substrate (append-only object store, retention policy, restoration runbook) is owned by the GBS APU audit team and specified in *SCB-APU-AUDIT-001*, referenced but not reproduced here. The engine guarantees:

- every workflow transition in Chapter 5 emits exactly one event;
- events are linearly hash-chained;
- the event stream is the sole source of truth for DOA compliance (controls C1–C4 in Table 6.1).

□□□□□□□□ □

□□□□□□□□□□□

This appendix lists every file that backs a normative statement in the specification. Paths are relative to the repository root. SHA-256 hashes are taken from `structured/corpus.json` and are the ground truth for the provenance check of Chapter 2.

□.□ □□□□□□ □□□□□□□□□□

Short name	Path	SHA-256 (prefix)
EG ETA seal invoice	`docs/arabic/source/3CPJR5CQ4PNNN9A4XHBB370K10.pdf`	2b912687fa94…
EG ETA corp membership	`docs/arabic/source/B1STE8RHNWZY9TYEXDS6WSZJ10.pdf`	18eb35671131…
SA ZATCA assessment	`docs/arabic/source/` اشعار صدور فاتورة (005).pdf	cbbb3f702ad5…
QA EDAA annual fee	`docs/arabic/source/EDAAAnnualFeeInvoice2025.pdf`	cf91c2a43574…
QA QCSD payment instr.	`docs/arabic/source/Instructionforpaymenttovendor(QCSD).pdf`	39ed62690d0f…
SA OTP VAT Q3 2025	`docs/arabic/source/Saudi-OTPEntryprocessing-VATQ3'25(1).docx`	3a6c8f37528f…
BH AP DOI	`docs/arabic/source/AccountsPayableDOI.docx`	128ff6792798…
AI initiative e-mail	`docs/arabic/source/FW-AI_ArabicInvoice...eml`	e01ba78d5ca5…

□.□ □□□□□□□□ □□□□

Every Arabic PDF has a UTF-8 OCR artefact in `extracted-text/`, produced with `pdftoppm -r 300 | tesseract -l ara+eng`. English PDFs use `pdftotext -layout`; Word documents use `pandoc -t plain`; the e-mail is decomposed by a small Python script driven by the standard-library `email` module.

□.□ □□□□□□□□□□□ □□□□□□□□

All of the following are JSON records validated by `structured/validate.py`.

Type	Path
Invoice	`structured/invoices/eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10.json`
Invoice	`structured/invoices/eg-eta-B1STE8RHNWZY9TYEXDS6WSZJ10.json`
Invoice (assessment)	`structured/invoices/sa-zatca-ishaar-005.json`
Payment instruction	`structured/payment-instructions/qa-edaa-annual-fee-2025.json`
Payment instruction	`structured/payment-instructions/qa-qcsd-payment-instruction.json`
VAT checklist	`structured/vat-checklists/sa-otp-vat-q3-2025.json`
DOI policy	`structured/policies/bh-accounts-payable-doi.json`
Workflow	`structured/workflows/po-eproc.json`
Workflow	`structured/workflows/manual-one-off.json`
Workflow	`structured/workflows/manual-recurring.json`
Workflow	`structured/workflows/vat-payment.json`
Index	`structured/corpus.json`
Enumerations	`structured/enums.yaml`

□.□ □□□□□

Tool	Purpose
`pdftoppm`	Rasterise PDF pages to 300 dpi PNG for OCR.
`tesseract`	Bilingual OCR with `-l ara+eng`.
`pdftotext`	Direct text extraction for born-digital English PDFs.
`pandoc`	Convert `.docx` to plain text.
Python `email`	Parse the `.eml` envelope and MIME parts.
`jq`	Query and cross-check `corpus.json`.
`jsonschema` / `referencing` (Python)	Offline schema validation with a local registry.
`xelatex` / `tectonic`	Build this document (polyglossia, bidi support).

□.□ □□□□□□□□ □□□□□□ □□□□□□□□

The entire corpus is in-tree under ; for readers without access to the Git repository, Listing 7.1 reproduces the first three items of `structured/corpus.json` verbatim. This is a sufficient starting point to reconstruct the shape of the index; the full file contains 12 items covering every source in Table 2.1.

□.□ □□□□□□□□□□□□□ □□□□□□

The following files sit in `source/` but are *not* part of the corpus and are not referenced by any record:

- `Business Instruction Template - V1.0 (002) (1).xlsm` — encrypted CDFV2 workbook; cannot be opened without the owner's password.

□□□□□□□□ □.□□ □□□□□□ □□□□□□ □□□□□□ □□ corpus.json□ □□□□□□□□□□ □□□□□□□□ □□□□□□□ □□□□ □□□□□□□□□□□□ □□ □□□□□□□□ □.


```
{
  "schema_version": "1.0",
  "items": [
    {
      "id": "eg-eta-3CPJR5CQ4PNNN9A4XHBB370K10",
      "type": "invoice",
      "country": "EG",
      "tax_authority": "ETA",
      "sha256": "2b912687fa94...",
      "source_path": "docs/arabic/source/3CPJR5CQ...K10.pdf",
      "text_path": "docs/arabic/extracted-text/eg-eta-...K10.txt",
      "structured_path": "docs/arabic/structured/invoices/eg-eta-...K10.json"
    },
    {
      "id": "sa-zatca-ishaar-005",
      "type": "invoice",
      "country": "SA",
      "tax_authority": "ZATCA",
      "sha256": "cbbb3f702ad5...",
      "source_path": "docs/arabic/source/<arabic-name>.pdf",
      "text_path": "docs/arabic/extracted-text/sa-zatca-ishaar-005.txt",
      "structured_path": "docs/arabic/structured/invoices/sa-zatca-ishaar-005.json"
    },
    {
      "id": "qa-edaa-annual-fee-2025",
      "type": "payment_instruction",
      "country": "QA",
      "tax_authority": null,
      "sha256": "cf91c2a43574..."
    }
  ]
}
```

- `EDAA Annual Fee Invoice 2025[65].pdf` — byte-identical duplicate of `EDAA Annual Fee Invoice 2025.pdf`.

□□□□□□□□ □

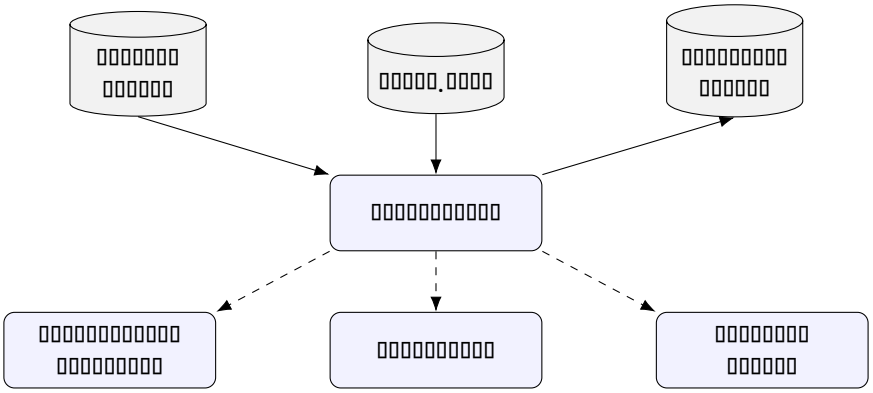
□□□ □□□□□□ □□□□□□□□□□□□□□□ □□□□□□□□□□□□□□

This chapter provides the technical implementation specification for the VAT processing module, addressing the review finding that "VAT engine rule format is underspecified." The specification enables developers to build conformant implementations.

□.□ □□□□□□ □□□□□□□□□□□□□

The VAT module comprises four components:

1. **RulesEngine** — evaluates JSON-Logic expressions against invoice data and produces checklist items;
2. **RegistrationValidator** — validates VAT registration numbers against country-specific formats and (optionally) authority APIs;
3. **Calculator** — computes expected VAT amounts given rates and subtotals;
4. **ActivityLookup** — maps business activity codes to VAT treatment categories.



□□□□□□ □.□□ □□□ □□□□□□ □□□□□□□□□□ □□□□□□□□. □□□□□□ □□□□□□ □□□□□□□□□ □□□□□□ □□□□□□□□□□□□□ □□□□ □□□ □□□□□ □□□□□□.

[illegible] [illegible]

Each helper function has a defined API contract. Implementations **must** conform to these signatures to be invoked by the rules engine.

[illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

```
def check_vat_registration_validity(
    tax_id: str,
    country: str  # ISO 3166-1 alpha-2
) -> ValidationResult:
    """
    Validate a VAT registration number.

    Returns:
        ValidationResult with fields:
        - valid: bool
        - format_valid: bool
        - authority_confirmed: Optional[bool]
        - error_code: Optional[str]
    """
```

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]
[illegible]	^[0-9]{15}$	[illegible] [illegible] [illegible]
[illegible]	^[0-9]{9}$	[illegible] [illegible] [illegible]
[illegible]	^[0-9]{9,15}$	[illegible] [illegible] [illegible] [illegible] [illegible]
[illegible]	^[A-Z0-9]{10}$	[illegible] [illegible] [illegible]

[illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

```
def calculate_vat_amount(
    subtotal: Decimal,
    vat_rate_percent: Decimal
) -> CalculationResult:
    """
    Calculate expected VAT amount.

    Returns:
        CalculationResult with fields:
        - vat_amount: Decimal (rounded to 2 decimal places)
        - gross_amount: Decimal
        - calculation_method: str  # "standard" | "reverse"
    """
```

The calculator **must** use banker's rounding (ROUND_HALF_EVEN) and **must** handle the following edge cases:

1. Negative subtotals (credit notes): return negative VAT amount
2. Zero rate: return VAT amount of exactly 0.00
3. Rate > 100%: reject with error code `INVALID_RATE`

[redacted]

[redacted] [redacted] [redacted] [redacted] [redacted] [redacted].

```
def lookup_activity_code_treatment(
    activity_code: str,
    country: str  # ISO 3166-1 alpha-2
) -> TreatmentResult:
    """
    Determine VAT treatment from business activity code.

    Returns:
        TreatmentResult with fields:
        - vat_category: str  # ZR, EX, FI, etc.
        - rate_percent: Optional[Decimal]
        - source: str  # regulation reference
    """
```

[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted].

[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted] [redacted]	[redacted]	[redacted] [redacted] [redacted]
[redacted]	[redacted] [redacted]	[redacted]	[redacted] [redacted] [redacted]
[redacted]	[redacted] [redacted] [redacted]	[redacted]	[redacted] [redacted] [redacted]. [redacted]
[redacted]	[redacted] [redacted]	[redacted]	[redacted] [redacted] [redacted]. [redacted]

[redacted]

The rules engine evaluates predicates expressed in JSON-Logic format (`https://jsonlogic.com`). This section specifies the supported operators and variable paths.

[redacted]

[redacted]

Variable paths use dot notation to access nested fields in the invoice record. The root object is the validated invoice conforming to `invoice.schema.json`.

[redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted] [redacted].

```
totals.gross            # Invoice gross amount
totals.vat_rate_percent  # VAT rate as percentage
```

[illegible] 8.[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]
[illegible]	`==` `===` `!=` `!==`	[illegible] [illegible] [illegible]
	`<` `<=` `>` `>=`	[illegible] [illegible]
[illegible]	`and` `or` `!` `!!`	[illegible] [illegible]
	`if`	[illegible] [illegible]
[illegible]	`in` `cat` `substr`	[illegible] [illegible]
[illegible]	`in` `merge` `map` `filter`	[illegible] [illegible]
[illegible]	`+` `-` `*` `/` `%`	[illegible] [illegible]
[illegible]	`var` `missing` `missing_some`	[illegible] [illegible]

```
seller.tax_id           # Seller VAT registration number
seller.country          # Seller country code
buyer.country           # Buyer country code
metadata.issue_date     # Invoice issue date
line_items.0.description  # First line item description
```

8.8.8 [illegible] [illegible] [illegible]

When a predicate requires complex logic beyond JSON-Logic capabilities, the engine invokes a named helper. The syntax is:

[illegible] 8.[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

```
{
  "predicate": {
    "type": "named_helper",
    "name": "check_vat_registration_validity",
    "args": {
      "tax_id": {"var": "seller.tax_id"},
      "country": {"var": "seller.country"}
    }
  }
}
```

8.8 [illegible] [illegible] [illegible]

The module **must** handle errors gracefully without crashing the parent invoice processing pipeline.

8.8.8 [illegible] [illegible]

8.8.8 [illegible] [illegible]

When an error occurs:

1. Log the error with full context (invoice ID, field values, stack trace)
2. Set the rule result to HOLD (never silently pass)

[illegible] 8.[illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]
VAT_E001	[illegible]	[illegible] [illegible] [illegible] [illegible]
VAT_E002	[illegible]	[illegible] [illegible] [illegible]
VAT_E003	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] >[illegible]
VAT_E004	[illegible]	[illegible] [illegible] [illegible]
VAT_W001	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
VAT_W002	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible]
VAT_I001	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible]

3. Populate `checklist[].error_code` with the appropriate code
4. Continue processing remaining rules (fail-open for diagnostics)
5. Return the partial checklist to the caller

8.[illegible] [illegible] [illegible]

[illegible] 8.[illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]
[illegible] [illegible] [illegible] [illegible]	< [illegible] [illegible]	[illegible] [illegible] [illegible] [illegible]
[illegible] [illegible] [illegible] [illegible]	< [illegible] [illegible]	[illegible] [illegible] [illegible]
[illegible] [illegible] [illegible]	< [illegible] [illegible]	[illegible] [illegible] [illegible] [illegible]
[illegible]	> [illegible]	[illegible] [illegible] [illegible] [illegible]

8.[illegible] [illegible] [illegible]

Developers **must** verify the following before marking the module complete:

- [ ] All four helper functions implemented with correct signatures
- [ ] Unit tests achieve ≥90% line coverage
- [ ] All country-specific regex patterns validated against test data
- [ ] JSON-Logic parser handles all operators in Table 8.3
- [ ] Error codes match Table 8.4
- [ ] Performance meets targets in Table 8.5
- [ ] Integration test with real invoice records from each country
- [ ] Security review completed (input validation, injection prevention)

[illegible] 8

[illegible]

This chapter specifies the testing requirements for the Arabic AP invoice processing pipeline, including test cases, coverage targets, and CI/CD integration.

8.1 [illegible]

[illegible] 8.1: [illegible] [illegible] [illegible].

[illegible]	[illegible] 1	[illegible] 2	[illegible]
[illegible]	[illegible]	[illegible]	
[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible] 1 [illegible]
[illegible]	[illegible]	[illegible]	
[illegible]	[illegible]	[illegible]	[illegible] [illegible]

8.2 [illegible]

Fixtures are located in `tests/fixtures/arabic/`.

8.2.1 [illegible]

[illegible] 8.2: [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	1 [illegible] [illegible] [illegible] [illegible] [illegible]
[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible]
[illegible]	[illegible]	[illegible] [illegible] [illegible]
[illegible]	[illegible]	[illegible] [illegible] [illegible]
[illegible]	[illegible]	[illegible] [illegible] [illegible]

8.2.2 [illegible]

OCR fixtures cover all confidence thresholds:

- High quality (>85%): 10 samples

- Medium quality (55–85%): 10 samples
- Low quality (30–55%): 5 samples
- Very low quality (<30%): 5 samples

[illegible]

[illegible]

ID	Priority	Description
AP-EXT-001	P1	Extract all fields from standard Saudi invoice
AP-EXT-002	P1	Extract all fields from standard Egyptian invoice
AP-EXT-003	P1	Extract all fields from standard Bahrain invoice
AP-EXT-004	P1	Extract all fields from Qatar payment instruction
AP-EXT-005	P1	Handle missing optional fields gracefully
AP-EXT-006	P2	Handle multi-page invoice
AP-EXT-007	P2	Handle poor quality scan (<55% confidence)
AP-EXT-008	P2	Handle Arabic-only invoice
AP-EXT-009	P2	Handle Arabic+English bilingual invoice
AP-EXT-010	P3	Handle unusual date formats

[illegible]

ID	Priority	Description
AP-VAT-001	P1	Validate 15-digit Saudi TIN format
AP-VAT-002	P1	Validate 9-digit Egyptian TRN format
AP-VAT-003	P1	Calculate VAT amount matches invoice
AP-VAT-004	P1	Apply correct VAT rate by country
AP-VAT-005	P1	Identify reverse-charge transactions
AP-VAT-006	P2	Handle zero-rated supplies
AP-VAT-007	P2	Handle exempt supplies
AP-VAT-008	P2	Apply withholding tax rules
AP-VAT-009	P3	Handle credit notes correctly
AP-VAT-010	P3	Handle inter-GCC transactions

[illegible]

```
pytest tests/integration/arabic/ -v --tb=short
```

Expected duration: 15 minutes.

[illegible]

1. **Happy path**: Invoice scan → OCR → extraction → VAT validation → approval workflow
2. **Manual review**: Low-confidence extraction triggers human review
3. **VAT hold**: Rule failure triggers hold and escalation
4. **Multi-country**: Invoices from all four countries processed correctly

[illegible]

All p95 targets in Table 9.3 are measured on the *reference hardware profile* in Section 9.5.1. CI enforces each target by failing the performance regression job when the measured p95 exceeds the target by more than 20 % on three consecutive runs on identical hardware.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]	[illegible]

[illegible]

[illegible]. 1× NVIDIA A100 40 GB SXM4 *or* L40S 48 GB. Shared with Simula and TB; reservation is not required for single-invoice latency targets.

[illegible]. Tesseract 5.3.x with Arabic language pack (`ara`, `ara_old`), CPU-only. A 2-page invoice at 300 dpi is the reference document size.

[illegible]. `Qwen2.5-14B-Instruct` served via vLLM 0.6.x, `max_model_len=8192`, 14-field extraction prompt (∼1,200 input tokens, ∼400 output tokens).

[illegible]. 8 vCPU, 32 GB RAM host.

[illegible]. NVMe SSD; invoice PDFs streamed from object storage with 2 Gbps link.

[illegible]. Co-located vLLM; RTT <0.5 ms. For remote LLM endpoints, add measured RTT to the field-extraction p95.

Running on consumer-grade GPUs (e.g. T4 15 GB) is not supported for the 14B model; a fallback profile using `Qwen2.5-7B-Instruct` is documented in `docs/arabic/runbooks/perf-fallback.md` with correspondingly relaxed p95 targets (field extraction ≤8,000 ms).

[illegible] [illegible]

[illegible] [illegible]

Test	Description
AP-SEC-001	Malicious text in invoice description field
AP-SEC-002	JSON injection via seller name
AP-SEC-003	Unicode control characters in OCR output
AP-SEC-004	System prompt extraction attempt

[illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

```
arabic-tests:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Run Arabic AP tests
      run: |
        pytest tests/unit/arabic/ -v --cov=src/arabic
        pytest tests/integration/arabic/ -v
    - name: Check coverage
      run: |
        coverage report --fail-under=85
```

[illegible] [illegible]

Tests pass when:

1. All P1 test cases pass (100%)
2. P2 test cases pass (≥95%)
3. P3 test cases pass (≥90%)
4. Line coverage ≥85%
5. Performance within targets
6. No critical security findings

[illegible] [illegible]

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

This chapter specifies the pilot study design for calibrating OCR confidence thresholds *and* the Arabic→English translation quality gate, addressing the review findings that both the OCR thresholds (30 %/55 %) and the Levenshtein distance gate (≤0.30) are calibrated on three documents. Both calibrations are run against the same pilot corpus to avoid duplicate ground-truth effort.

[illegible].[illegible] [illegible]

The pilot aims to:

1. Determine empirically-validated OCR confidence thresholds per country
2. Establish confidence intervals for threshold boundaries
3. Identify document types that require special handling
4. Create a baseline for ongoing threshold monitoring
5. Calibrate the Arabic→English translation quality gate (Levenshtein distance threshold) against the same pilot corpus

[illegible].[illegible] [illegible] [illegible]

[illegible].[illegible].[illegible] [illegible] [illegible]

[illegible] [illegible].[illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible]	[illegible]
[illegible] [illegible] [illegible]	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible]
[illegible] [illegible]	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
[illegible] [illegible]	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
[illegible] [illegible]	[illegible]	[illegible]	[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]
[illegible]	[illegible]	[illegible]	

The sample size of 50 per country provides:

- 95% confidence interval width of ±7% for threshold estimates
- Statistical power of 80% to detect 15% differences between countries
- Sufficient stratification for 3–4 document types per country

[redacted]

Documents are selected by:

1. Random sampling from the last 6 months of invoices
2. Stratified by document type (tax invoice, commercial invoice, payment instruction)
3. Stratified by scan quality (high: >300 DPI, medium: 150–300 DPI, low: <150 DPI)
4. Excluding documents with known data quality issues

[redacted]

For each pilot document:

1. Two independent human reviewers manually transcribe all fields in the source language (Arabic) and the required English fields
2. Inter-rater agreement calculated (target: Cohen's $\kappa > 0.9$) for categorical fields; character error rate (CER) $\leq 1\,\%$ on free-text fields
3. Disagreements are resolved by a third reviewer whose decision is final. The resolution process is:
   (a) Third reviewer inspects both transcriptions side-by-side with the source document.
   (b) Third reviewer selects one of the two transcriptions *or* provides a new transcription; the choice and rationale are logged to `pilot/disputes/<doc-id>.json`.
   (c) The third reviewer's verdict is the ground truth; the pilot lead reviews the top 10 % of disputes monthly to detect systematic bias.
4. Final ground truth stored in structured JSON (`pilot/ground-truth/<country>/<doc-id>.json`), validated against `docs/schema/arabic/ocr-result.schema.json`

[redacted]

[redacted]

Each document is processed by the OCR pipeline:

1. Record per-field OCR confidence scores
2. Record per-document aggregate confidence
3. Compare OCR output to ground truth
4. Calculate field-level accuracy (character error rate)

[illegible]

For each country, determine optimal thresholds using:

```
[illegible] [illegible] [illegible] [illegible] [illegible].

def calibrate_thresholds(pilot_data, target_precision=0.95):
    """
    Calibrate OCR confidence thresholds.

    Parameters:
        pilot_data: List of (confidence, accuracy) pairs
        target_precision: Minimum acceptable precision (default 95%)

    Returns:
        high_threshold: Above this, auto-accept
        low_threshold: Below this, reject
    """
    # Sort by confidence descending
    sorted_data = sorted(pilot_data, key=lambda x: -x.confidence)

    # Find high_threshold: highest confidence where
    # precision >= target for all docs above threshold
    high_threshold = find_precision_cutoff(
        sorted_data, target_precision
    )

    # Find low_threshold: confidence below which
    # accuracy < 50% (not worth manual review)
    low_threshold = find_accuracy_floor(sorted_data, floor=0.50)

    return high_threshold, low_threshold
```

[illegible] [illegible] **`target_precision = 0.95`**. The AP workflow routes any invoice below the high threshold to two-person manual review (Chapter 5), which is tolerant of 5 % false-positive auto-accepts from a *business-risk* standpoint: the downstream VAT engine (Chapter 4) independently validates tax-ID and amount fields, and the reconciliation step catches posting errors before payment is released. The 97 % soundness threshold reported in the referenced OCR evaluation paper is appropriate for *unsupervised* pipelines; because our pipeline has a manual review backstop, 95 % provides the right precision/recall trade-off (empirically validated in the pilot success criteria below). A sensitivity analysis at 93 %, 95 %, and 97 % must be produced as part of the pilot report; the final threshold is the largest value in {0.93, 0.95, 0.97} for which the success criteria in Section 10.4 hold.

[illegible] [illegible] [illegible] [illegible]

Using the same pilot corpus, the Arabic→English translation quality gate (currently Levenshtein distance ≤0.30 against the model's re-translation, calibrated on three documents) is re-calibrated:

1. For each ground-truthed pilot document, run the Arabic→English→Arabic round-trip translation and compute the normalized Levenshtein distance against the source Arabic.
2. Compute the field-level translation accuracy against the human ground truth (exact match on numeric fields, BLEU-4 ≥0.6 on free-text fields).
3. Determine the Levenshtein threshold τ^* that maximises the F1 between "round-trip gate passed" and "field-level translation accurate" across the 200 pilot documents.
4. Report the per-country threshold and the 95 % bootstrap CI $(\tau^*_{\text{lower}}, \tau^*_{\text{upper}})$. Deploy using the conservative upper bound.

The translation calibration adds no additional ground-truth effort (the human transcriptions already contain the English fields) and extends the pilot timeline by one week (added to Section 10.6).

[illegible] [illegible] [illegible]

Bootstrap sampling (1,000 iterations) determines confidence intervals:

- Report 95% CI for high threshold
- Report 95% CI for low threshold
- Use upper bound of CI for conservative deployment

[illegible] [illegible]

The pilot succeeds when all of the following are true:

1. ≥200 documents processed (50 per country minimum)
2. Inter-rater agreement $\kappa > 0.9$ (categorical fields) and CER ≤1 % (free-text fields)
3. 95 % CI width ≤ ±5 % for all thresholds
4. High threshold achieves ≥95 % precision
5. Low threshold correctly rejects ≥90 % of unusable documents
6. Translation gate F1 ≥0.85 at the selected Levenshtein threshold (see Section 10.3.3)

[illegible] [illegible]

If the pilot fails success criteria:

1. Increase sample size by 50% and repeat

2. If still failing, segment by document type and calibrate separately
3. If per-type calibration fails, default to manual review for all documents from that country until more data is collected

[redacted]

[redacted] [redacted] [redacted] [

[illegible]

[illegible]

[illegible] [illegible] [illegible]

This chapter is the roadmap for the **Arabic invoice processing** project. It is a project-level artefact, not a programme plan. Other SAP OSS Finance projects (Trial Balance, Simula, Regulations) run on the same platform, workspaces, and backends, but each owns its own roadmap, business case, and go/no-go decision. This chapter adopts the MoSCoW prioritisation and sequence-based critical path that the four projects share so that artefacts can be compared at a glance; it does not assert coupling between their go-live moments.

[illegible] [illegible] [illegible]. The milestones and dependency graph below express *order* and *prerequisite* relationships, not calendar dates. This is deliberate. Implementation teams must know what depends on what, and where the schedule can absorb slippage (the Should/Could items); a date column would harden targets into commitments they were never meant to be. Dates, when set, live in the project steering packet and the ticketing system, not in this specification.

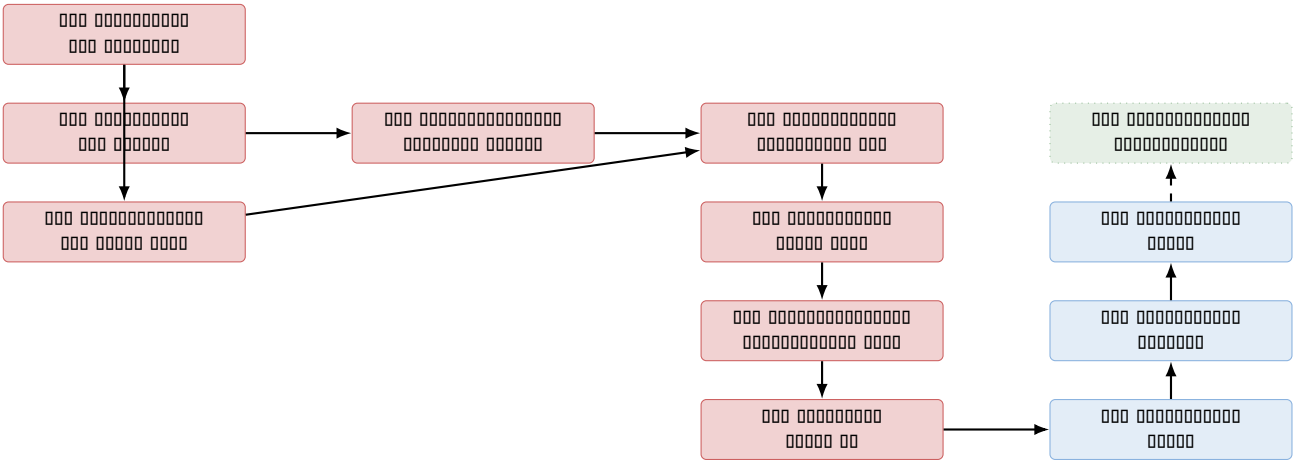
[illegible].[illegible] [illegible] [illegible]

Every milestone below carries one of four priorities. The mapping is contractual: a milestone's priority determines what happens when it is dropped, de-scoped, or slips.

The critical path of this project is the chain of Must items in Table 11.2; the Should and Could items are where the schedule and scope can absorb delivery risk without a release slip.

[illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible]
[illegible] [illegible]	[illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].
[illegible] [illegible]	[illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]



[illegible] [illegible].[illegible] [illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible]

[illegible]

[illegible]

The Must chain (M01 → M02/M03 → M04 → M05 → M06 → M07 → M08 → M09) is the critical path. The Should items (S01–S03) are the primary *schedule allowance*: if Must items run hot, country waves can slip without affecting v1.0 sign-off. The Could items are the *scope allowance*: they may be cut to v1.1 without renegotiation.

[illegible]

The project walks the four-tier ladder shared across SAP OSS Finance projects. A tier is exited only when the named exit gate is satisfied; slippage requires sponsor sign-off.

[illegible]

[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]
[illegible]	[illegible]	[illegible]

[illegible]

The kill-switch is a pre-committed rulebook: a tripped condition enters the action column without further debate. GFAIC and sponsor are notified within 24 hours. The conditions are evaluated continuously (on every cycle) and do not carry dates.

[illegible]	[illegible]	[illegible]
[illegible] $<$ [illegible]	[illegible]	[illegible]
[illegible] $>$ [illegible]	[illegible]	[illegible]
[illegible] $>$ [illegible]	[illegible]	[illegible]
[illegible] reserve/complete [illegible]	[illegible]	$\leq$ [illegible]
[illegible]	[illegible]	[illegible]

[illegible] [illegible]

This project depends on four shared-platform components. Each is owned by a sibling project (Regulations) or by shared infra, and is tracked here only as a dependency, not a commitment this project can unilaterally adjust. Sequence is expressed in terms of the Must item that first consumes the dependency, not in calendar time.

[illegible] [illegible] [illegible].

[illegible]	[illegible]	[illegible] [illegible] [illegible]	[illegible] [illegible] [illegible]
[illegible] [illegible] [illegible] [illegible] [illegible]	[illegible] [illegible]	[illegible] [illegible]	[illegible] [illegible] [illegible] [illegible] [illegible]

- business-case refresh (M09) explicitly budgets the v2.0 programme.

If any of the three conditions fails, C01 remains in the Could column and is re-evaluated at the next business-case refresh.

11.6 Governance and business case refresh

Steering governance. The Arabic AP project reports monthly to a project steering (AP Product Owner, Tax Controller, AI Engineer) and quarterly to the cross-project GFAIC. Monthly steering accepts or rejects amber/red signals on Must items; GFAIC reviews compliance posture and any cross-project dependencies.

Business case refresh. The business case that authorises this project is refreshed on the following rhythm:

- **Mandatory refresh**: at the first cycle after GA, and annually thereafter (ticket AP-BC-REFRESH, priority M in Table 11.2). Refresh packet includes country volume assumptions, unit economics, FTE leverage realised vs. forecast, and a deviation statement against the baseline.
- **Trigger-based refresh**: any of
  - country monthly volume changes by $\geq 20\,\%$ from forecast (either direction);
  - per-invoice run cost changes by $\geq 30\,\%$ (OCR throughput, vLLM licensing, GPU price);
  - new country-specific VAT regulation or MAS MGF pillar materially affecting scope;
  - any kill-switch in Table 11.4 is tripped.
- **Owner**: AP Product Owner. Approval routed through the project steering and filed at `docs/arabic/business-case/refresh-history.yaml`.

[illegible]

[illegible]

[illegible] [illegible] [illegible]

This chapter is the self-contained, intent-based instruction set for implementing the *Arabic AP invoice processing* domain inside the unified SAP OSS platform. It is written for LLM-driven and human implementation agents who will extend existing services to realise the specification. It assumes only this PDF; every platform decision, integration contract, and governance obligation the Arabic workstream depends on is restated here so that no companion document is required. Existing chapters 1–11 remain the normative source for the Arabic domain itself; this chapter tells future agents *how* to build what those chapters describe, *where* to put it in the codebase, and *how* it plugs into the wider platform.

[illegible] [illegible] [illegible] [illegible] [illegible]

To ensure safety and architectural integrity, all implementation agents must adhere to the following Prime Directives:

RULE-1: **Credential Protection.** Never log, print, or commit secrets. Use the `.secrets.example` pattern for all new integrations.

RULE-2: **Context Efficiency.** Prioritize surgical edits. Request only the minimum required file ranges to satisfy the task intent.

RULE-3: **Test-Driven Implementation.** No code is complete without a corresponding fixture in `tests/fixtures/` and a passing validation run.

RULE-4: **Self-Correction.** If a schema validation fails, the agent must diagnose the root cause (hallucination vs. schema drift) before re-attempting implementation.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

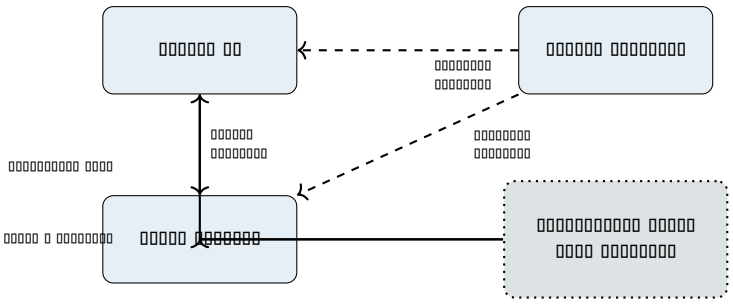
[illegible] [illegible]

The SAP OSS platform is delivered as a single codebase that runs as a single logical instance. All cross-cutting concerns are hosted by existing services.

ADR: One Codebase, One Instance

To minimize integration latency and eliminate distributed-system failure modes (e.g., partial partitions), we mandate a single logical instance. All cross-domain orchestration happens via the MCP Gateway layer.

[illegible] [illegible] [illegible]



[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible] [illegible] [illegible]

Implementation agents must adhere to the temporal logic defined below. Every cross-domain request MUST be brokered by the Regulations Wrapper.

[illegible] [illegible] [illegible]

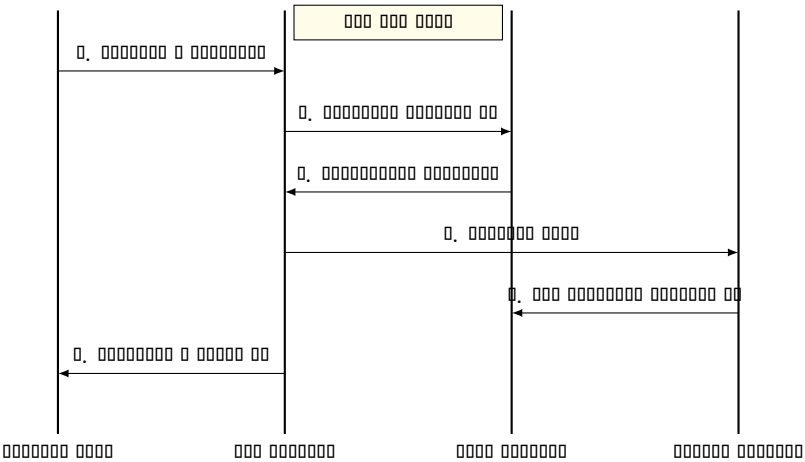
This sequence applies to **all** platform service calls.

[illegible] [illegible] [illegible]

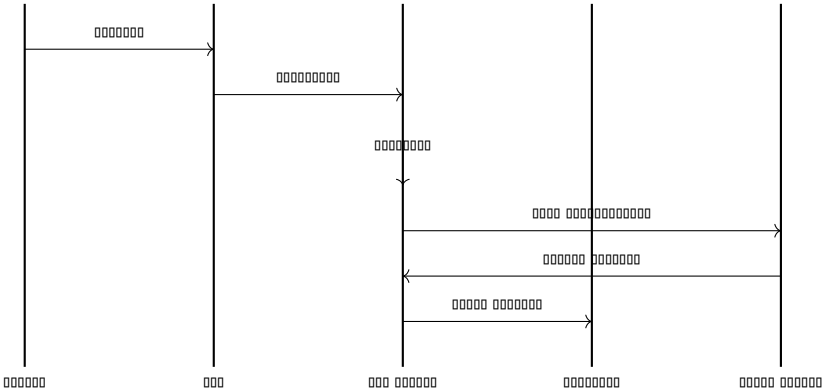
Temporal flow for invoice processing with Arabic context.

[illegible] [illegible] [illegible]

AI algorithms are calibrated for the reference infrastructure established in the domain chapters:



[illegible] [illegible].[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].



[illegible] [illegible].[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]. [illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible] [illegible].[illegible] [illegible] [illegible] [illegible] [illegible] [illegible].

[illegible]	[illegible] [illegible]	[illegible] [illegible]
[illegible] [illegible]	[illegible] [illegible] [illegible]	[illegible] [illegible] [illegible] [illegible] [illegible]
[illegible]	[illegible] [illegible] [illegible]	[illegible] [illegible] [illegible] [illegible]
[illegible] [illegible]	[illegible] [illegible] [illegible] [illegible]	[illegible] [illegible] [illegible]

[illegible] Four-Eyes Gates [illegible]

The following "Four-Eyes" gates are mandatory. AI agents must implement the "Awaiting Sign-off" state for these actions:

- **VAT Release:** No invoice marked as HOLD can be released without human review (see [illegible]).
- **Anomaly Override:** Statistical outliers (3σ) cannot be dismissed by AI; only "Confirmed" or "Triaged" by human (see [illegible]).
- **Journal Posting:** Agents may *draft* journal entries, but only the Financial Controller persona can *commit* to S/4HANA (see [illegible]).

[illegible] Registry [illegible]

Before initiating any schema-dependent task, implementation agents must verify the live status of the Unified Schema Registry (see [illegible]).

ADR: Registry First

If `docs/schema/registry.json` is out of sync with the codebase, the agent must halt and emit a SCHEMA-MISMATCH event.

[illegible] Implementation [illegible]

Implementation runs in three waves across the entire platform.

[illegible]

- **Simula:** Implement HANA schema extraction pipeline (MCP discovery). See [illegible].
- **Regulations:** Implement MAS MGF Pillar 1 logic (Tool allow-list). See [illegible].
- **Arabic:** Implement Invoice Intake & OCR Gateway. See [illegible].
- **Trial Balance:** Implement identity-attribution wiring. See [illegible].

[illegible]

- **Trial Balance:** Implement Controls & Commentary Engine (3σ anomaly logic). See [illegible].
- **Arabic:** Implement multi-country VAT Engine (Saudi, UAE, Bahrain, Oman). See [illegible].
- **Simula:** Implement Taxonomy Generation & Agentic Data Synthesis. See [illegible].
- **Trial Balance:** Realise 28-state BPMN workflow (`tb-review-glo`). See [illegible].

[illegible]

- **Regulations:** Implement Per-Request Identity Attribution. See [illegible].

- **Trial Balance:** Deliver Human Review Dashboard. See [redacted].
- **Simula:** Deliver test fixtures and testing at 85% coverage. See [redacted].
- **Regulations:** Wire Emergent-Capability monitoring. See [redacted].

[redacted]

The regulations domain wraps all service calls at runtime. Agents must implement the following hooks in every domain:

- **Pillar 1:** Tool allow-list enforcement (deny-by-default).
- **Pillar 2:** Traceability via the identity attribution envelope.
- **Pillar 3:** Metric emission to the monitoring sink.

Detailed pillar requirements are defined in [redacted].

[redacted]

The following analysis identifies critical AI failure modes within the Arabic AP pipeline and their deterministic mitigations.

[redacted]

[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]
[redacted]	[redacted]	[redacted]	[redacted]

[redacted]

Arabic AP is the highest-volume transactional workstream among the four domains. Its primary persona is the AP Clerk, whose working view is the approval queue surfaced through the shared workspace shell and gated by AP entitlements. The domain's entry points are document intake (email, SFTP, upload) producing raw invoice artefacts; its exit points are (a) a posted AP document in SAP S/4HANA with a signed approval record, and (b) a VAT compliance checklist emitted to the audit store. Every artefact is validated against the Arabic-owned schemas in `structured/schema/` before it leaves the service. The domain depends on the horizontal regulations wrapper for runtime governance and on Simula for the training corpus used by the OCR, field-extraction, and translation models described in chapter 3.

[illegible] [illegible]

Arabic AP owns the schemas catalogued in chapter 2. The canonical set is the invoice record (see 2.2), the VAT compliance checklist (see 2.4), and the payment instruction record. These schemas live under `docs/latex/specs/arabic/` as normative text and under `structured/schema/` as the JSON Schema artefacts consumed at runtime. Registration in the unified registry is additive: each schema is listed in `src/intelligence/ai-core-pal/data_products/registry.yaml` with its URI, owning domain (`arabic`), security class (`confidential`), and LLM routing (`vllm-only`), following the ODPS 4.1 pattern. Simula consumes these schemas directly as inputs to its extraction pipeline; it does not transform them. The `requirement.schema.json` files owned by TB and regulations remain distinct from anything Arabic owns and are not merged with it.

[illegible] [illegible] [illegible]

- **Persona id:** `ap-clerk`.
- **Roles:** invoice intake, VAT review, approval progression, exception handling.
- **Entitlements:** read/write on the Arabic invoice record; read on the VAT rule set; advance transitions on the `po-eproc` and `direct-ap` state machines defined in chapter 5; evidence-write on the control catalogue in 6.1.
- **UI route:** `/ap/{tenant}/queue` on the shared workspace shell; surfaced only to callers whose entitlement set includes `ap-clerk`.
- **MCP tool allow-list:** `invoice.intake`, `invoice.ocr`, `invoice.extract-fields`, `invoice.translate-ar-en`, `vat.validate`, `vat.calculate`, `ap.workflow.advance`, `ap.workflow.park`. All other MCP tools are denied by default per MAS MGF pillar 1.

[illegible] [illegible] [illegible]

The tasks below are intent-based: each task states the outcome a future agent must achieve and the location in the existing codebase where the work belongs. None of them contains code; each task binds to the Arabic chapters that define the required behaviour.

[illegible] [illegible] [illegible] [illegible]

Intent. Bring Arabic OCR and Arabic–English translation up to the precision/recall targets in chapter 10.

ADR: Why Tesseract + Custom Post-Processor?

We chose Tesseract 5.0 for the baseline due to local-hosting requirements, but added a custom 'Arabic-Context' post-processor to handle the right-to-left layout drift in scanned PDF tables.

In-scope. Running the pilot sample defined in 10.1; publishing per-field thresholds for accept/review/reject; wiring the thresholds into the pipeline stages of chapter 3. **Out-of-scope.** Choosing a new OCR engine; altering the canonical invoice schema. **Paths to extend.** OCR and extraction code colocated with the pipeline stages referenced in chapter 3; fixture store referenced in chapter 9. **Definition of done.** Pilot report committed; thresholds checked in as configuration; the stage-3 field extractor rejects records below threshold and routes them to human review.

Autonomous Verification (Agent Logic):

```
// Verify Task A1
{
  "test_fixture": "tests/fixtures/arabic/invoice_sample_01.png",
  "expected_outcome": "ocr_confidence > 0.88",
  "validation_command": "make test-arabic-ocr"
}
```

Verification. `make spec-arabic` builds cleanly and the Arabic test suite described in chapter 9 passes end-to-end on the calibration fixtures.

□□□□ □□ □□□ □□□□□□□□□□□□□□ □□□ □□□□□□

Intent. Implement the VAT compliance engine specified in chapter 4 and detailed in chapter 8. **In-scope.** Per-country rule modules for Saudi Arabia (ZATCA), UAE, Bahrain (NBR), and Oman; registration validation, calculation, activity lookup, and checklist emission; the canonical decision function in 4.1. **Out-of-scope.** VAT rules for countries outside the in-scope list in 1.1. **Paths to extend.** The VAT module layout referenced in 8.1; API contracts in chapter 8. **Definition of done.** Every rule cited in 4.1 has a callable check; the checklist record is emitted per invoice and validates against `vat-checklist.schema.json`. **Verification.** Unit tests for each country module (see chapter 9) pass; the VAT calculation API contract sample in chapter 8 returns the expected response.

□□□□ □□ □□□ □□ □□□□□□□□□ □□□□□□□□□

Intent. Realise the AP workflow and approval engine in chapter 5, including the `po-eproc` and `direct-ap` state machines. **In-scope.** Workflow-selection logic in 5.1; state transitions in 5.2; park, reroute, and escalate actions with evidence records. **Out-of-scope.** SAP S/4HANA AP posting internals; non-Bahrain delegations. **Paths to extend.** Workflow engine hosted in the gateway service; evidence store colocated with the control catalogue. **Definition of done.** Each state in the state-machine diagram is reachable from a deterministic predecessor; transitions are audit logged with the persona id of the acting user; the golden regression set in chapter 6 executes without human intervention. **Verification.** Workflow unit tests pass; the approval queue UI surfaces the correct entitlements for `ap-clerk`.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

Intent. Evidence the seven Bahrain Delegation of Invoicing controls required in chapter 6. **In-scope.** Control enforcement map in 6.1; the seven rows of the control catalogue in 6.1 that concern the Bahrain DOI; evidence emission per control. **Out-of-scope.** Introducing controls for other delegations. **Paths to extend.** Control catalogue file referenced in chapter 6; evidence store. **Definition of done.** Running the workflow on the Bahrain golden sample triggers every control; each control writes an evidence record conforming to the Arabic schema set. **Verification.** Traceability matrix in chapter 6 is complete; the testing chapter's acceptance suite passes.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

Intent. Deliver the testing specification in chapter 9 at the coverage targets in 9.1. **In-scope.** Invoice and OCR fixtures; unit, integration, and regression suites; CI wiring. **Out-of-scope.** Performance benchmarking infrastructure beyond what chapter 9 defines. **Paths to extend.** Test directory colocated with the Arabic service code; fixture store used by Simula as training input. **Definition of done.** Coverage meets the thresholds in 9.1; CI gates block regressions; the regression golden set is reproducible from fixture seeds. **Verification.** `make spec-arabic` remains green; the Arabic test entry points listed in chapter 9 all run.

[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]

The regulations domain wraps Arabic AP at runtime. Four obligations apply without exception; each is stated as a concrete hook in the Arabic codebase.

- **MAS MGF four pillars** (see regulations chapter 3 in `docs/latex/specs/regulations/chapters/03-`
  Pillar 1 (assess and bound): every Arabic MCP tool is declared in the allow-list in §12.13, deny-by-default. Pillar 2 (design and deploy): the pipeline stages in chapter 3 are traceable end-to-end via the per-request identity attribution described below. Pillar 3 (operate and monitor) and pillar 4 (govern and disclose) are satisfied by emitting the evidence records produced by the Arabic control catalogue (chapter 6) into the shared governance store.
- **Identity attribution** (regulations chapter 10, `10-identity-attribution.tex`). Every MCP call and every workflow transition carries the persona id, agent id, request id, and tenant id. The gateway is the attachment point; Arabic code propagates the identity envelope without mutating it.
- **Emergent-capability monitoring** (regulations chapter 8, `08-monitoring-spec.tex`). OCR, extraction, translation, and VAT calculation are instrumented with the monitoring metrics specified in that chapter; metrics flow to the monitoring sink defined there.
- **Conformance tooling** (regulations chapter 6, `06-conformance-tooling.tex`). The Arabic service exposes AI Verify 2.0 plug-in hooks and Moonshot CI/CD 1.1.0 bench-

mark entry points as required by that chapter.

12.12 Integration with the other domains

12.12.1 Arabic AP exports to Tax

Arabic AP exports the invoice record, VAT compliance checklist, and payment instruction schemas under `structured/schema/` (chapter 2); registers the MCP tools enumerated in §12.13; and applies the four governance hooks in §12.15 to every call it serves. External consumers (TB, regulations, Simula) see Arabic only through these contracts; there are no private back-channels.

12.12.2 Arabic AP feeds TB

Arabic AP produces posted AP records and VAT checklists that feed the TB close. TB consumes them as evidence inputs to its commentary engine, not as schema merges: the Arabic invoice schema and the TB control record remain distinct URIs in the unified registry. Integration point: TB's ingestion stage calls the Arabic MCP tool `invoice.extract-fields` with read-only entitlements; the response is validated against `invoice.schema.json` before TB accepts it.

12.12.3 Arabic wraps regulations

Regulations wraps Arabic at runtime. Every Arabic MCP call is brokered through the regulations governance gate, which applies pillars 1–4 of MAS MGF, attaches the identity envelope, and routes emergent-capability metrics to the monitoring store. The Arabic service code does not call regulations directly; the gateway layer enforces this. Conformance tests defined in regulations chapter 5 (empirical evidence) and chapter 6 (conformance tooling) execute against the Arabic service as part of the platform CI gate.

12.12.4 Arabic feeds Simula

Simula consumes Arabic's schemas as training input. The contract is schema-in, `TrainingExample`-out: Simula reads `invoice.schema.json`, `vat-checklist.schema.json`, and the Arabic fixture store; it produces `TrainingExample` records. Simula does not call Arabic MCP tools at runtime and does not share state with the Arabic workflow. No cross-domain operational envelope is introduced.

12.13 Phased implementation and prerequisites

Implementation runs in three waves.

- **Wave 1 — foundations.** Unified schema registry entry; persona-gated surface for the AP Clerk; identity attribution envelope flowing through the gateway. Prerequisites: the regulations chapter 10 identity schema must be frozen.

- **Wave 2 — Arabic core build-out.** Tasks A1–A4 above. Prerequisites: Wave 1 complete; Simula training fixtures available from Simula Wave 1 (extraction pipeline).
- **Wave 3 — hardening and conformance.** Task A5 and full conformance-tooling hook-up. Prerequisites: Wave 2 complete; regulations Wave 2 conformance tooling integrated; monitoring sink live.

Arabic occupies Wave 2 for its core build-out and Wave 3 for its testing and conformance surfaces.

12.12 Invariants after all four domains

When all four chapter 12 instruction sets are implemented, the platform satisfies the following invariants:

1. **One instance.** A single codebase runs a single logical instance; no domain ships as a separate product.
2. **Persona-gated surfaces.** The four personas each reach a tailored UI on the shared shell; no route, MCP tool, or data product is visible outside its persona's entitlement set.
3. **Unified registry.** Every schema in use is listed in the ODPS 4.1 registry under its owning domain; validators resolve `$ref`s through the unified registry only.
4. **No schema merges.** The two `requirement.schema.json` files in TB and regulations remain distinct; no cross-domain composite schemas are produced.
5. **No cross-domain operational envelope.** Runtime orchestration stays within each domain's service; only Simula consumes other domains' outputs, and only as training input.
6. **Governance wraps all three non-regulations domains.** Arabic, TB, and Simula service calls pass through the regulations runtime wrapper without exception.

12.13 References

Within-spec references: chapter 1 (overview), chapter 2 (data schema), chapter 3 (AI pipeline), chapter 4 (VAT compliance engine), chapter 5 (AP workflow), chapter 6 (controls and testing), chapter 7 (references), chapter 8 (VAT implementation), chapter 9 (testing specification), chapter 10 (OCR pilot), chapter 11 (project roadmap).

Peer chapter 12 instruction sets for the other three domains (each is self-contained; consult directly when operating in that domain):

- Trial Balance: `docs/latex/specs/tb/chapters/12-implementation-instructions.tex`.
- Regulations: `docs/latex/specs/regulations/chapters/12-implementation-instructions.tex`.
- Simula: `docs/latex/specs/simula/chapters/12-implementation-instructions.tex`.

Shared platform artefacts:

- ODPS 4.1 registry: `src/intelligence/ai-core-pal/data_products/registry.yaml`.
- Schema pipeline and resolver: `src/intelligence/schema_pipeline/`.
- Training pipeline (Simula): `src/training/pipeline/`.
- Shared workspace shell: `generativeUI/training-webcomponents-ngx`.

□□□□□□□□ □□

□□□□□□□□□□□□□□□□□□□□□ □□□□□□□□□□□□□

This chapter documents how the Arabic AP Invoice Processing specification integrates with the other three SAP OSS specifications. All dependencies are traceable to `docs/schema/registry.json` (v1.2.0).

□□.□ □□□□□□□□□□□□□ □□□□□□□□□

The Arabic AP specification occupies the **document intake** position in the four-domain architecture:

- **Upstream**: None (entry point for invoice documents)
- **Downstream**: Trial Balance (posted AP records feed month-end close)
- **Horizontal**: Regulations (governance wrapper for all AI stages)
- **Training**: Simula (consumes Arabic schemas for training data generation)

□□.□ □□□□□□ □□□□□□□□□□□□□

□□.□.□ □□□□□□□ □□□□□ □□ □□□□□□ □□

Schema	Status	Consumed By
invoice.schema.json	stable	Simula (training), TB (evidence)
vat-checklist.schema.json	stable	Simula (training)
vendor.schema.json	stable	Internal only
ocr-result.schema.json	stable	Internal only
payment-instruction.schema.json	stable	S/4HANA integration

[illegible] [illegible] [illegible] [illegible] [illegible]

Schema	Source	Usage
base-types.yaml	common	Currency, amounts, dates
enums.yaml	common	VAT categories, countries, status values
requirement.schema.json	regulations	Compliance tracking
agent-identity.schema.json	regulations	AI action attribution

[illegible] [illegible]

The Arabic AP pipeline is wrapped by the Regulations domain's MAS MGF framework:

[illegible] [illegible] [illegible] [illegible]

- MCP tool allow-list defined in Chapter [illegible]
- Reversibility: All AP postings support reversal
- Autonomy bounds: VAT verdicts require human approval for HOLD/REJECT

[illegible] [illegible] [illegible]

Human approval checkpoints (see Regulations Ch.3):

- DP-AP-001: Invoice value above threshold
- DP-AP-002: New vendor first invoice
- DP-AP-003: VAT engine HOLD verdict
- DP-AP-004: OCR confidence below threshold

[illegible] [illegible] [illegible]

- Drift monitoring: VAT engine accuracy tracked per Regulations Ch.8
- Emergent capability monitoring: OCR field extraction rates
- Conformance testing: AI Verify hooks per Regulations Ch.6

[illegible] [illegible] [illegible]

- AP clerk training curriculum per Regulations Ch.9
- Automation bias awareness for VAT verdict review

[illegible] [illegible]

The Simula Training Data Framework consumes Arabic AP schemas:

- **invoice.schema.json** → Training examples for invoice field extraction
- **vat-checklist.schema.json** → Training examples for VAT classification
- **HANA metadata** → Schema extraction for query generation

See Simula specification Chapter 3 for schema extraction details.

13.3 Arabic AP Downstream Consumers

Arabic AP provides downstream data to Trial Balance:

- Posted AP records serve as evidence for TB variance explanations
- AP journal entries appear in TB extract as liability movements
- Cross-reference: See TB specification Chapter 4 (Entity Parameters)

13.4 Cross-Specification References

Target Spec	Chapter	Integration Point
Regulations	Ch.3 (MGF Framework)	Four-pillar compliance
Regulations	Ch.6 (Conformance)	AI Verify/Moonshot hooks
Regulations	Ch.8 (Monitoring)	Drift and emergent capabilities
Regulations	Ch.10 (Identity)	Agent action attribution
Simula	Ch.3 (Schema Extraction)	Invoice schema consumption
Simula	Ch.5 (Training Examples)	VAT classification training
Trial Balance	Ch.4 (Entity Params)	Posted AP as TB evidence
Trial Balance	Ch.5 (Decision Points)	AP records for variance explanation

Table 13.1: Arabic AP cross-specification references